EV CHARGING STATION MANAGEMENT SYSTEM

PBCST304 OBJECT ORIENTED PROGRAMMING
PROJECT REPORT

submitted by

<Student Name 1> (<University Reg. No. 1>)
<Student Name 2> (<University Reg. No. 2>)
<Student Name 3> (<University Reg. No. 3>)

to

The APJ Abdul Kalam Technological University

in partial completion of the criteria for the degree award of

B.Tech Degree

in

COMPUTER SCIENCE AND ENGINEERING



SCMS SCHOOL OF ENGINEERING AND TECHNOLOGY (AUTONOMOUS)
(Affiliated to APJ Abdul Kalam Technological University)
VIDYA NAGAR, PALISSERY, KARUKUTTY ERNAKULAM – 683582

October 2026

DECLARATION

We, the undersigned, hereby declare that the project report entitled **"EV CHARGING STATION MANAGEMENT SYSTEM"**, submitted for partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering of the APJ Abdul Kalam Technological University, Kerala, is a bonafide work done by us under the supervision of **Mr. Anoop Jose**, Assistant Professor, Department of Computer Science and Engineering.

This submission represents our ideas in our own words, and where ideas or words of others have been included, we have adequately and accurately cited and referenced the original sources. We also declare that we have adhered to the ethics of academic honesty and integrity and have not misrepresented or fabricated any data, idea, fact or source in our submission. We understand that any violation of the above will be a cause for disciplinary action by the institute and/or the University and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been obtained.

This report has not previously formed the basis for the award of any degree, diploma or similar title of any other University or Institute.

Signatures:

1. _____
<Student Name 1> (<Reg. No. 1>)

2. _____
<Student Name 2> (<Reg. No. 2>)

3. _____
<Student Name 3> (<Reg. No. 3>)

Place: Ernakulam
Date: 27 October 2026

SCMS SCHOOL OF ENGINEERING AND TECHNOLOGY (AUTONOMOUS)
(Affiliated to APJ Abdul Kalam Technological University)
VIDYA NAGAR, PALISSERY, KARUKUTTY ERNAKULAM – 683582

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the report entitled **"EV CHARGING STATION MANAGEMENT SYSTEM"**, submitted by **<Student Name 1> (<Reg. No. 1>)**, **<Student Name 2> (<Reg. No. 2>)**, and **<Student Name 3> (<Reg. No. 3>)** to the APJ Abdul Kalam Technological University in partial fulfilment of the requirements for the award of the Degree of Bachelor of Technology in Computer Science and Engineering is a bonafide record of the project work carried out by them under my/our guidance and supervision.

This report in any form has not been submitted to any other University or Institute for any purpose.

Project Co-ordinator

Mr. Anoop Jose
Assistant Professor
Department of CSE
SSET, Karukutty

Head of the Department

Dr. Manish T.I
Professor
Department of CSE
SSET, Karukutty

ACKNOWLEDGEMENT

We are greatly indebted to **Dr. Jenson Joseph E.**, Principal, SCMS School of Engineering and Technology, Ernakulam, and **Dr. Manish T.I**, Professor and Head of the Department, Department of Computer Science and Engineering, SCMS School of Engineering and Technology, who wholeheartedly permitted us to conduct this project.

We would like to express our deepest sense of gratitude and sincere thanks to our project co-ordinator, **Mr. Anoop Jose**, Assistant Professor, Department of Computer Science and Engineering, SCMS School of Engineering and Technology, who gave us valuable guidance, critical suggestions, continuous encouragement, and technical advice throughout the duration of this software engineering work.

We would like to express our sincere gratitude to all the faculty members of the Department of Computer Science and Engineering who gave us their constant moral and technical support during various phases of software requirement analysis, object-oriented system modeling, relational schema architecture, and software verification.

We would also like to thank the supporting staff in the computer laboratory whose dedicated work and technical assistance kept the laboratory working smoothly, thus enabling us to have continuous access to development workstations, software development kits, database servers, and testing tools required to execute and validate the project.

Finally, we extend our heartfelt appreciation and gratitude to our beloved parents, family members, and friends for their enduring patience, moral encouragement, and necessary resources that empowered us to successfully complete this academic project.

ABSTRACT

The rapid global transition toward sustainable e-mobility has accelerated the adoption of Electric Vehicles (EVs). However, public charging infrastructure faces critical operational challenges, including physical connector incompatibilities, inefficient station scheduling, prolonged queue wait times, lack of emergency priority dispatching, non-transparent billing, and unverified customer feedback systems. This project, **EV Charging Station Management System (EvChargingApp)**, develops a robust, full-stack, enterprise-grade desktop software platform built using **Java (JDK 21)**, **Java Swing GUI**, **JDBC**, and a relational **MySQL 8.0** database engine to resolve these systemic bottlenecks.

The application is architected strictly adhering to **Object-Oriented Programming (OOP)** design principles, enforcing a clear separation of concerns across a four-tier architecture consisting of Domain Entities (Model layer), Data Access Objects (DAO layer), Business Logic and Services (Service layer), and a modern dark-themed Graphical User Interface (Presentation layer). A key technological contribution of the platform is the custom dynamic priority queue engine implemented in `BookingQueue.java`, which utilizes a comparator-driven dispatching algorithm that immediately prioritizes emergency charging requests over routine charging sessions, while dynamically resolving tie-breaks among normal requests by dispatching vehicles with the lowest State of Charge (battery percentage) first.

Furthermore, the system incorporates an automated physical connector compatibility validation engine that verifies vehicle inlet compatibility (CCS2, Type 2, CHAdeMO, and GB/T) against station charger specifications before booking persistence, thereby preventing mechanical connector mismatches and electrical hazards. Real-time telemetry emulation tracks continuous energy consumption (kWh), delivering automated, transparent billing based on station tariff rates. Multi-method financial settlement supports UPI, Credit/Debit Cards, Digital Wallets, and Cash, tied directly to an automated customer loyalty rewards ledger and a verified post-session review audit mechanism. The software system has been rigorously validated through an automated test suite comprising 21 programmatic unit and integration assertions, ensuring complete functional robustness, transaction reliability, and relational integrity.

TABLE OF CONTENTS

Chapter	Title	Page No.
	Acknowledgement	i
	Abstract	ii
	List of Tables	iv
	List of Figures	v
1	**INTRODUCTION**	**1**
	1.1 Introduction to EV Charging Management Systems	1
	1.2 Background and Motivation	2
	1.3 Problem Statement	3
	1.4 Objectives of the Project	4
	1.5 Scope of the Project	5
	1.6 Significance of the Project	6
2	**EXISTING AND PROPOSED SYSTEM**	**7**
	2.1 Existing System	7
	2.2 Limitations of the Existing System	8
	2.3 Proposed System	9
	2.4 Advantages of the Proposed System	10
	2.5 Methodology	11
3	**IMPLEMENTATION DETAILS**	**13**
	3.1 System Requirement Specification	13
	3.2 Hardware Requirements	14
	3.3 Software Requirements	14
	3.4 Programming Language and Technologies	15
	3.5 Platform and IDE	16
	3.6 Database and Database Connectivity	16
	3.7 Application Architecture and Layered Implementation	18
	3.8 Important Algorithms and Workflows	22
	3.9 Automated Verification and Testing Strategy	25
4	**SYSTEM DESIGN**	**27**
	4.1 System Architecture Diagram	27
	4.2 Use Case Diagram & Actor Descriptions	28
	4.3 Activity Diagram & Operational Workflows	30
	4.4 Data Flow Diagrams (DFD Level 0 & Level 1)	31

	4.5 Entity-Relationship (ER) Diagram & Relational Schema	32
	4.6 Database Design & Data Dictionary	33
	4.7 UML Class Diagram & Structural Relationships	37
5	**SCREENSHOTS**	**39**
	5.1 System Dashboard & Operations Overview	39
	5.2 Driver Registration & Vehicle Profile Management	40
	5.3 Charger Infrastructure & Availability Panel	41
	5.4 Booking Management & Priority Dispatch Queue	42
	5.5 Charging Session Monitor & Energy Metering	43
	5.6 Multi-Method Payment Gateway & Billing Audit	44
	5.7 Customer Ratings & Verified Review Management	45
	5.8 Database Diagnostics & Health Console	46
6	**CONCLUSION**	**47**
	6.1 Summary of the Project	47
	6.2 Key Achievements Based on Implementation	47
	6.3 Limitations	48
	6.4 Future Enhancements	48
7	**REFERENCES**	**49**

LIST OF TABLES

Table No.	Title	Page No.
Table 2.1	Comparative Analysis of Existing Systems vs. Proposed System	10
Table 3.1	Hardware Requirements Specification	14
Table 3.2	Software Requirements Specification	14
Table 3.3	Technology Stack and Framework Specifications	15
Table 3.4	Automated Test Suite Execution Results (AppTestSuite - 21 Assertions)	26
Table 4.1	Data Dictionary - DRIVERS Table	33
Table 4.2	Data Dictionary - VEHICLES Table	34
Table 4.3	Data Dictionary - CHARGING_LOCATIONS Table	34
Table 4.4	Data Dictionary - LOCATION_AMENITIES Table	35
Table 4.5	Data Dictionary - CHARGERS Table	35
Table 4.6	Data Dictionary - BOOKINGS Table	36
Table 4.7	Data Dictionary - CHARGING_SESSIONS Table	36
Table 4.8	Data Dictionary - PAYMENTS Table	36
Table 4.9	Data Dictionary - LOYALTY_TRANSACTIONS Table	37
Table 4.10	Data Dictionary - REVIEWS Table	37

LIST OF FIGURES

Figure No.	Title	Page No.
Figure 4.1	Multi-Tier Architecture Diagram of EV Charging Management System	27
Figure 4.2	UML Use Case Diagram showing Actors and Functional Use Cases	29
Figure 4.3	Activity Diagram for Booking Dispatching and Charging Workflow	30
Figure 4.4	Data Flow Diagram (DFD Level 1) for Core System Processes	31
Figure 4.5	Entity-Relationship (ER) Diagram and Relational Database Schema	32
Figure 4.6	UML Class Diagram detailing Entities, DAOs, Services, and UI Components	38
Figure 5.1	System Dashboard & Operations Overview	39
Figure 5.2	Driver Registration & Vehicle Profile Management	40
Figure 5.3	Charger Infrastructure & Real-Time Availability Panel	41
Figure 5.4	Booking Management & Priority Dispatch Queue	42
Figure 5.5	Charging Session Monitor & Energy Metering	43
Figure 5.6	Multi-Method Payment Gateway & Billing Audit	44
Figure 5.7	Customer Ratings & Verified Review Management	45
Figure 5.8	Database Diagnostics & Connectivity Health Console	46

CHAPTER 1
INTRODUCTION

1.1 Introduction to EV Charging Management Systems

The transition from internal combustion engine vehicles to Electric Vehicles (EVs) represents one of the most transformative technological paradigms in global transportation engineering. Driven by decarbonization goals, escalating fossil fuel costs, and environmental mandates to reduce greenhouse gas emissions, electric vehicle adoption has seen exponential expansion across domestic, commercial, and fleet sectors. However, the widespread viability and consumer confidence in electric mobility are fundamentally tethered to the availability, reliability, efficiency, and intelligence of EV charging infrastructure.

Unlike conventional gasoline refilling stations where fuel dispensing occurs mechanically within three to five minutes, electric vehicle charging is inherently a power-intensive, electronically governed process. Charging duration spans anywhere from twenty minutes to several hours depending on battery capacities, state-of-charge, power levels (AC Level 2 vs. DC Fast Charging), and vehicle-charger electrical compatibility. Consequently, the operational management of EV charging infrastructure requires sophisticated computing systems capable of orchestrating multi-connector hardware, dynamically managing customer waitlists, tracking live kilowatt-hour (kWh) energy telemetry, managing transparent tariff billing, executing secure multi-channel payments, and maintaining relational audit ledgers.

An EV Charging Management System (EVCMS) is an integrated software platform designed to supervise, automate, and streamline the daily operations of electric vehicle supply equipment (EVSE). An effective EVCMS bridges the physical charging station infrastructure with drivers and operational administrators. It transforms disconnected charging points into an intelligent, interconnected network that balances driver convenience, energy metering accuracy, prioritized queue dispatching, and secure financial settlements.

1.2 Background and Motivation

The commercial deployment of public and private charging stations in metropolitan regions and national highway corridors has highlighted severe operational challenges. Early charging infrastructure implementations operated as isolated, uncoordinated standalone terminals. Drivers frequently arrived at charging hubs only to find physical connector incompatibilities, out-of-service chargers, or extensive vehicle queues without any transparent booking or wait-time forecasting.

Furthermore, charging station queues typically operate on an indiscriminate First-Come, First-Served (FCFS) basis. While FCFS is straightforward to implement, it entirely disregards critical vehicular urgency. For instance, an emergency response vehicle, a medical transport EV, or an electric car stranded with less than 10% battery charge remaining is forced to queue behind vehicles that already have 75% charge and are merely conducting top-up charging. This structural deficiency induces range anxiety among drivers, reduces station operational throughput, and creates severe bottlenecks during peak travel hours.

From an administrative and commercial standpoint, early charging setups suffered from decentralized accounting, vulnerability to manual billing errors, lack of customer retention

incentives, and an absence of verified customer feedback mechanisms. Operators lacked unified dashboard consoles to observe live charging telemetry, inspect database connectivity health, and enforce automated business rules. These systemic shortcomings motivated the design and development of the **EV Charging Station Management System (EvChargingApp)**, an enterprise desktop platform engineered in Java and MySQL that unifies priority queue scheduling, automated compatibility checks, real-time energy metering, multi-method payment auditing, customer loyalty programs, and verified feedback into a cohesive software architecture.

1.3 Problem Statement

The operation and management of contemporary electric vehicle charging facilities are hindered by critical technological, operational, and data-integrity limitations:

1. **Physical Connector Incompatibility:** The electric vehicle market incorporates multiple incompatible physical inlet standards, including Combined Charging System 2 (CCS2), Type 2 (Mennekes AC), CHAdeMO, and GB/T. Manual or unverified bookings frequently allow drivers to reserve chargers physically incompatible with their vehicle inlet, resulting in wasted station reservation slots, driver frustration, and operational disruption.

2. **Inefficient and Inflexible Queue Dispatching:** Conventional FCFS queue scheduling lacks the algorithmic intelligence to distinguish between emergency travel demands and non-urgent top-ups, leading to critical service delays for stranded drivers and emergency fleet operators.

3. **Opaque Energy Metering and Tariff Calculation:** Legacy charging setups often utilize manual billing approximations or disconnected hardware meters, introducing accounting discrepancies and financial mistrust between drivers and station operators.

4. **Unstructured Payment Processing and Auditability:** Station operators often lack unified payment gateways capable of recording heterogeneous transaction modes (UPI, credit/debit cards, digital wallets, and cash) with strict relational foreign-key consistency and transactional audit trails.

5. **Absence of Verified Customer Feedback:** Most existing review mechanisms permit unverified or fraudulent ratings, depriving operators of actionable operational insights and misleading prospective drivers regarding station amenity quality and charger reliability.

1.4 Objectives of the Project

The primary objective of this project is to architect, develop, and validate an integrated, desktop-based EV Charging Station Management System that provides end-to-end management of EV drivers, vehicles, charging hardware, priority queues, charging sessions, financial billing, customer loyalty, and station reviews. The specific engineering objectives are:

• **To implement a Layered Enterprise Architecture:** Design a clean, decoupled four-tier architecture in Java utilizing Domain Entities (Model layer), Data Access Objects (DAO layer), Business Logic and Services (Service layer), and a modern Java Swing GUI (Presentation layer).

• **To engineer a Dynamic Priority Queue Dispatching Engine:** Construct a custom priority queue dispatching algorithm utilizing Java's `PriorityQueue` and a custom comparator that

prioritizes emergency bookings first, followed dynamically by vehicles with the lowest remaining battery percentage.

• **To enforce Automated Connector Compatibility Verification:** Build a business rule validation engine in the service layer that cross-references vehicle connector types against charger connector specifications before permitting booking persistence in the database.

• **To simulate Real-Time Charging Telemetry and Metering:** Develop a live session metering simulator that records energy consumption (kWh) and automatically calculates accurate billing amounts based on charger-specific tariffs.

• **To implement an Auditable Multi-Method Payment and Loyalty Subsystem:** Provide multi-channel financial settlement (UPI, Card, Wallet, Cash) with relational database foreign-key enforcement, automatic loyalty points accrual (1 point per $\text{INR } 10$ spent), and points redemption.

• **To build a Verified Feedback and Review Mechanism:** Enforce strict business constraints permitting review and star rating submissions exclusively on completed charging bookings.

• **To validate System Reliability via Programmatic Testing:** Develop an automated test suite (`AppTestSuite.java`) executing unit, integration, and database assertions to verify algorithm correctness, lifecycle state machines, and relational data integrity.

1.5 Scope of the Project

The scope of the EV Charging Station Management System encompasses the functional and technical boundaries implemented within the desktop application:

• **User and Fleet Management:** Registration and maintenance of EV drivers, contact profiles, linked electric vehicles, battery capacities (in kWh), connector types, and live battery state-of-charge.

• **Station Infrastructure Administration:** Cataloging of charging stations, geographical locations, operational amenities (e.g., Wi-Fi, cafes, restrooms), individual charger units, charging modes (Fast, Ultra-Fast, Normal), connector standards, tariff rates per kWh, and real-time availability toggling.

• **Booking and Dispatching Management:** Request creation with Normal and Emergency flags, automated connector compatibility checks, priority queue management, and booking state transitions.

• **Session Telemetry and Energy Billing:** Real-time session monitoring, start/complete session state machine, energy calculation (kWh), and automated invoice generation.

• **Financial Transactions and Loyalty Ledger:** Settlement across multiple payment modes, automated loyalty reward points calculation, and transactional ledger maintenance.

• **Database Health Diagnostics:** Integrated database connection testing, configuration management via `db.properties`, table row inspection, and diagnostics console.

Out of Scope: The current release focuses on a robust desktop-based management application for station operators and terminal kiosks. Direct hardware-level embedded micro-controller firmware communication (such as low-level CAN bus or physical Open Charge Point Protocol

OCPP v2.0.1 over WebSockets) and mobile native applications (Android/iOS) are defined as future enhancements.

1.6 Significance of the Project

The development of this EV Charging Station Management System is highly significant from both academic and industrial perspectives. Academically, it serves as a comprehensive demonstration of core Computer Science principles, including Object-Oriented Software Design, Data Structures (custom comparator-based priority heaps), Design Patterns (DAO, MVC, Singleton, Factory), relational database design with referential integrity and check constraints, and GUI human-computer interaction principles.

Industrially, the system addresses critical pain points in commercial EV charging infrastructure. By introducing intelligent emergency-first and low-battery priority scheduling, the system minimizes vehicle stranding risks and maximizes station resource utilization. By automating connector compatibility verification, it eliminates human error and scheduling conflicts. The transparent billing, loyalty program, and verified review mechanisms establish an auditable, trustworthy ecosystem for both EV drivers and charging network operators.

CHAPTER 2
EXISTING AND PROPOSED SYSTEM

2.1 Existing System

In the existing ecosystem of electric vehicle charging management, operational practices vary from manual logbooks and isolated spreadsheet tracking to fragmented, proprietary kiosk applications. In many municipal and private installations, chargers function as disconnected, standalone hardware terminals. Drivers navigate to stations without prior knowledge of connector availability or compatibility, leading to physical queuing at station gates.

Where software solutions exist, they frequently operate as basic scheduling calendars that apply strict First-Come, First-Served (FCFS) queuing. Such systems treat all charging requests identically regardless of the vehicle's remaining battery percentage or emergency status. Driver and vehicle profile data are frequently stored in disconnected tables without referential constraints, resulting in orphaned records, missing contact details, and inaccurate battery specifications.

Furthermore, existing legacy setups handle financial settlement through disconnected point-of-sale (POS) terminals or cash receipts that are not synchronized with charging telemetry. Billing calculations rely on manual data entry by station attendants, which introduces human calculation errors and vulnerability to revenue leakage. Customer feedback is either non-existent or collected via unauthenticated guestbooks that allow arbitrary reviews without verifying whether the reviewer actually completed a charging session.

2.2 Limitations of the Existing System

The limitations of conventional and existing EV charging management approaches include:

1. **Rigid FIFO Scheduling:** Existing systems rely on simple FIFO queues that cannot prioritize vehicles with critical charge depletion ($<20\%$) or emergency utility vehicles, leading to increased risk of battery breakdown and traffic bottlenecks.

2. **Manual or Absent Compatibility Verification:** Conventional software allows reservations based solely on charger availability without checking whether the vehicle inlet matches the charger connector (e.g., allowing a CCS2 vehicle to reserve a Type 2 AC outlet), resulting in aborted sessions.

3. **Disjointed Energy Metering and Billing:** Energy consumption data (kWh) from the charging unit is not directly linked to billing records, leading to billing discrepancies, delayed invoicing, and lack of auditability.

4. **Lack of Unified Multi-Payment Integration:** Fragmented payment methods force operators to manually reconcile cash, UPI, and card payments against daily sessions, increasing administrative burden and accounting errors.

5. **Absence of Customer Retention and Loyalty Architecture:** Traditional stations treat transactions as one-off events without structured customer reward points or retention programs.

6. **Unverified and Fraudulent Customer Feedback:** Public reviews cannot be verified against actual charging transactions, allowing malicious negative reviews or fraudulent positive ratings that distort station reputation.

7. **Poor Architectural Modularity:** Many existing applications are written as monolithic spaghetti scripts that tightly couple UI code with database SQL queries, rendering system maintenance, unit testing, and component expansion challenging.

2.3 Proposed System

The proposed **EV Charging Station Management System (EvChargingApp)** provides an integrated, robust, and extensible desktop solution designed using clean software engineering methodologies. The proposed system replaces disparate manual workflows with an automated, multi-tier software architecture that coordinates the entire charging lifecycle from driver registration to post-charge review submission.

At the core of the proposed system is an intelligent priority queue scheduling subsystem implemented in `BookingQueue.java`. The queue evaluates incoming charging requests using a custom comparator that instantly prioritizes bookings flagged as `EMERGENCY` over standard bookings. For non-emergency bookings, the algorithm dispatches vehicles with the lowest battery percentage first, ensuring that vehicles near battery exhaustion are serviced before vehicles with ample remaining charge.

The proposed system integrates an automated compatibility validation engine within `Charger.java` and `BookingService.java`. Before any booking is persisted, the system verifies that the vehicle's connector type matches the charger's physical standard. Real-time telemetry emulation continuously tracks energy delivered (kWh), dynamically updating session state and automatically calculating total payable amounts based on active station tariff rates. Multi-method financial settlement (UPI, Card, Wallet, Cash) is integrated with an automated loyalty reward points ledger (1 point per $\text{INR } 10$ spent) and a verified customer review system restricted exclusively to completed charging bookings.

Table 2.1: Comparative Analysis of Existing Systems vs. Proposed System

Evaluation Parameter	Existing / Legacy Systems	Proposed EvChargingApp System
Queue Scheduling	Strict First-Come, First-Served (FCFS); no urgency awareness.	Dynamic Priority Queue: Emergency bookings first, then lowest battery percentage.
Connector Verification	Manual or absent; frequent mechanical connector mismatches.	Automated pre-booking compatibility check (CCS2, Type 2, CHAdeMO, GB/T).

Evaluation Parameter	Existing / Legacy Systems	Proposed EvChargingApp System
Energy Metering & Billing	Manual meter reading and manual invoice calculation.	Automated session telemetry (kWh) and tariff rate multiplication.
Payment Processing	Fragmented, unlinked POS receipts or manual cash logs.	Unified multi-mode payment (UPI, Card, Wallet, Cash) with relational audit records.
Loyalty Program	Non-existent or disconnected physical coupon cards.	Automated relational loyalty ledger (1 pt per $\text{INR } 10$) with audit trail.
Customer Reviews	Unverified public reviews; vulnerable to manipulation.	Strict verification: Ratings allowed only on completed charging sessions.
Software Architecture	Monolithic scripts mixing SQL, UI, and business rules.	Clean 4-tier OOP architecture: Model, DAO, Service, and Swing Presentation layers.
Testing & Verification	Manual ad-hoc testing without regression safety.	Automated test suite (`AppTestSuite.java`) executing 21 test assertions.

2.4 Advantages of the Proposed System

The proposed system provides substantial operational, financial, and technical advantages:

• **Optimized Station Throughput & Reduced Stranding:** The priority dispatching algorithm ensures emergency vehicles and severely depleted batteries receive immediate charging access, minimizing vehicle breakdown risk.

• **Error-Free Booking Reservations:** Automated connector validation eliminates reservation errors and mechanical incompatibility aborts.

• **Transparent and Accurate Financial Auditability:** Direct binding between energy delivered (kWh), station tariff rates, session status, and payment records guarantees complete transparency and eliminates billing disputes.

• **Customer Retention via Gamified Loyalty:** Automatic accrual and redemption of loyalty points incentivizes recurring driver visits and brand loyalty.

• **Authentic Reputation Metrics:** Restricting review submissions to completed bookings ensures that station ratings reflect genuine driver experiences.

• **Maintainable and Scalable Codebase:** Adherence to Object-Oriented design patterns (DAO, Service, Model, View) facilitates rapid feature expansion, code reusability, and automated regression testing.

2.5 Methodology

The development of the EV Charging Station Management System adhered to an iterative software engineering methodology governed by Object-Oriented Analysis and Design (OOAD). The engineering lifecycle comprised the following phases:

1. **Requirement Engineering & Domain Modeling:** Comprehensive identification of functional and non-functional requirements, followed by domain modeling of real-world charging entities (Driver, Vehicle, ChargingLocation, Charger, Booking, ChargingSession, Payment, Review).

2. **Relational Database Architecture:** Formulation of a normalized (3NF) relational database schema in MySQL 8.0 containing 10 interdependent tables with primary keys, foreign key constraints (with cascading deletions where appropriate), unique indexes, and column-level check constraints.

3. **Layered Application Implementation:** Construction of the Java codebase across four discrete packages: `com.evcharge.app.model` for domain POJOs, `com.evcharge.app.dao` for JDBC persistence, `com.evcharge.app.service` for core business logic, and `com.evcharge.app` for Swing GUI presentation and application themes.

4. **GUI Human-Computer Interface Design:** Development of an ergonomic, dark-themed Java Swing desktop interface featuring sidebar navigation, responsive card panels, tabular data views, KPI metric counters, real-time telemetry indicators, and modal diagnostics.

5. **Automated Programmatic Verification:** Engineering of `AppTestSuite.java` containing 21 automated assertions testing priority queue sorting, connector compatibility, loyalty math, state transitions, review validation rules, and live MySQL connectivity.

CHAPTER 3
IMPLEMENTATION DETAILS

3.1 System Requirement Specification

The System Requirement Specification (SRS) establishes the foundational functional and non-functional capabilities of the EV Charging Station Management System.

3.1.1 Functional Requirements

• **FR1: Driver & Vehicle Management:** The system shall allow registration, updating, and retrieval of EV drivers (name, phone, email, loyalty points) and associated vehicles (model, battery capacity in kWh, connector type, and current battery percentage).

• **FR2: Charging Infrastructure Management:** The system shall manage charging locations, amenities, and individual chargers, tracking charger codes, charging modes (Fast/Ultra-Fast/Normal), connector standards, tariff rates, and availability.

• **FR3: Physical Connector Compatibility Verification:** The system shall automatically validate that a vehicle's connector type matches the target charger's connector standard before allowing booking confirmation.

• **FR4: Dynamic Priority Queue Dispatching:** The system shall maintain a prioritized queue that dispatches emergency charging requests first, and resolves normal requests based on ascending battery percentage (lowest battery first).

• **FR5: Charging Session & Energy Metering:** The system shall manage charging session lifecycles (`NOT_STARTED`, `CHARGING`, `COMPLETED`), recording start time, end time, energy delivered (kWh), and calculated total billing amount.

• **FR6: Multi-Method Payment Processing:** The system shall process payments across UPI, Card, Wallet, and Cash modes, persisting transaction status (`PENDING`, `PAID`, `FAILED`) and payment timestamps.

• **FR7: Customer Loyalty Accrual and Redemption:** The system shall calculate loyalty reward points at the rate of 1 point per $\text{INR } 10$ spent on paid sessions, updating driver balances and recording transaction audit records.

• **FR8: Verified Rating and Review Submission:** The system shall validate that customer ratings (1 to 5 stars) and textual feedback are accepted exclusively for bookings that have reached the `COMPLETED` status.

3.1.2 Non-Functional Requirements

• **NFR1: Performance & Response Time:** Local database queries and UI transitions shall execute in under 500 ms.

• **NFR2: Data Integrity & Consistency:** Relational integrity shall be strictly enforced via foreign keys, unique constraints, and check constraints across all 10 MySQL tables.

• **NFR3: Usability & Ergonomics:** The desktop interface shall employ consistent typography, high-contrast dark theme styling, clear color-coded status badges, and feedback toasts for all user actions.

• **NFR4: Robustness & Error Handling:** Database connection failures, SQL exceptions, and invalid inputs shall be handled gracefully with descriptive error dialogues and console diagnostics without application crashes.

• **NFR5: Maintainability:** The software shall strictly decouple presentation, business logic, data persistence, and entity definitions to support future extension and unit testing.

3.2 Hardware Requirements

Table 3.1: Hardware Requirements Specification

Component	Minimum Requirement	Recommended Specification
Processor (CPU)	Intel Core i3 / AMD Ryzen 3 (Dual Core 2.0 GHz)	Intel Core i5 / AMD Ryzen 5 or higher (Quad Core 2.5+ GHz)
Random Access Memory (RAM)	4 GB DDR4	8 GB / 16 GB DDR4
Hard Disk Space	500 MB free disk space	2 GB SSD storage
Display Resolution	1280×720 pixels	1920×1080 Full HD
Input Devices	Standard Keyboard and Mouse / Trackpad	Standard Keyboard and Mouse / Touchscreen terminal

3.3 Software Requirements

Table 3.2: Software Requirements Specification

Software Component	Specification / Version
Operating System	Microsoft Windows 10 / Windows 11 (64-bit), Ubuntu Linux 22.04 LTS, or macOS 13+
Java Development Kit (JDK)	Oracle OpenJDK 21 / Oracle JDK 21 LTS
Database Management System	MySQL Community Server 8.0 or 9.x
Database Driver	MySQL Connector/J 9.1.0 (JDBC Driver)
Integrated Development Environment	Eclipse IDE for Java Developers (2024-09 or newer) / IntelliJ IDEA
Build & Dependency Tool	Apache Maven 3.9+ / Eclipse Built-in Compiler

3.4 Programming Language and Technologies

The application is engineered exclusively with modern, standards-compliant development technologies:

Table 3.3: Technology Stack and Framework Specifications

Technology	Role and Implementation Detail
Java Standard Edition (Java SE 21)	Core programming language providing modern language features, strong type safety, robust collections framework (PriorityQueue, Streams), and object-oriented paradigms.
Java Swing (javax.swing & java.awt)	Desktop GUI toolkit utilizing CardLayout for sidebar navigation, customized table renderers, dark-mode styling, and responsive event handling.

Technology	Role and Implementation Detail
Java Database Connectivity (JDBC)	Standardized API for thread-safe database communication, parameterized PreparedStatements, transaction boundaries, and ResultSet processing.
MySQL Server 8.0+	Enterprise-grade relational database engine enforcing relational constraints, foreign keys with cascading deletions, and ACID compliance.
Apache Maven	Project build automation, standardized directory layout, compiler lifecycle management, and artifact dependency resolution (`pom.xml`).

3.5 Platform and IDE

The development, debugging, and packaging of the application were executed within the **Eclipse IDE for Java Developers** on the Windows 64-bit platform. Eclipse provided comprehensive integration with the Java compiler (ecj), automated classpath management (via `.classpath`), integrated Git source versioning, and GUI thread debugging.

Project dependencies and build automation are managed through `pom.xml`, which specifies source and target compilation compliance for Java 21, links the MySQL Connector/J JDBC driver library (version 9.1.0), and packages executable JAR artifacts. Configuration settings and database credentials are fully externalized within `db.properties`, enabling runtime adaptability across local development workstations and deployment servers without modifying source code.

3.6 Database and Database Connectivity

3.6.1 Database Schema & Table Structures

The underlying relational database schema is named `ev_charging_app` and is encoded with the `utf8mb4` character set and `utf8mb4_0900_ai_ci` collation. The database schema encompasses 10 tables designed to 3NF standards:

1. **`drivers`:** Manages driver primary keys, full name, unique telephone number, unique email address, and accumulated loyalty points.

2. **`vehicles`:** Records electric vehicles associated with drivers, storing model name, battery capacity (kWh), connector standard (CCS2, Type 2, CHAdeMO, GB/T), and current battery percentage with a check constraint (`0 ≤ pct ≤ 100`).

3. **`charging_locations`:** Stores station properties, geographical addresses, location categories (Hub, Cafe, Highway), verification status, and aggregate ratings (0.0 to 5.0).

4. **`location_amenities`:** Catalogs amenities (Wi-Fi, Restrooms, Cafes) linked to charging locations with cascading deletions.

5. **`chargers`:** Stores individual charging terminals linked to locations, capturing unique charger codes, charging mode (Fast/Ultra-Fast/Normal), connector standard, price per kWh, and live availability flags.

6. **`bookings`:** Records reservation requests, linking driver, vehicle, and charger, capturing request type (`NORMAL`, `EMERGENCY`), battery percentage at request, status state machine, and timestamps.

7. **`charging_sessions`:** Represents active and completed charging events, tracking start/end timestamps, energy delivered (kWh), calculated billing amount, and session status (`NOT_STARTED`, `CHARGING`, `COMPLETED`).

8. **`payments`:** Maintains payment transactions linked to completed sessions, recording payment method (`UPI`, `CARD`, `WALLET`, `CASH`), settled amount, status (`PENDING`, `PAID`, `FAILED`), and payment timestamps.

9. **`loyalty_transactions`:** An audit ledger recording loyalty points earned or redeemed per payment transaction with positive integer check constraints.

10. **`reviews`:** Stores verified driver feedback, capturing 1 to 5 star ratings and driver commentary linked to verified booking IDs.

3.6.2 Connection Management & JDBC Implementation

Database communication is centralized within `DBConnection.java`. The class utilizes the Singleton and Factory design patterns to initialize and distribute thread-safe JDBC database connections. To prevent hardcoded credentials and facilitate flexible deployment, `DBConnection` implements a resilient two-tier property loader:

```
public class DBConnection {
    private static String url = "jdbc:mysql://localhost:3306/ev_charging_app?...";
    private static String user = "root";
    private static String password = "password";

    static {
        loadProperties();
    }

    private static void loadProperties() {
        Properties props = new Properties();
        boolean loaded = false;
```

```
        // Priority 1: Load from classpath
        try (InputStream is = DBConnection.class.getClassLoader()
                .getResourceAsStream("db.properties")) {
            if (is != null) { props.load(is); loaded = true; }
        } catch (Exception ignored) {}

        // Priority 2: Fallback to file system path
        if (!loaded) {
            try (InputStream is = Files.newInputStream(Paths.get("db.properties")))
{
                props.load(is); loaded = true;
            } catch (Exception ignored) {}
        }
        if (loaded) {
            url = props.getProperty("db.url", url);
            user = props.getProperty("db.user", user);
            password = props.getProperty("db.password", password);
        }
    }

    public static Connection getConnection() throws SQLException {
        return DriverManager.getConnection(url, user, password);
    }
}
```

This implementation guarantees that the application seamlessly adapts to both Eclipse runtime classpaths and standalone command-line executions without configuration overhead.

3.7 Application Architecture and Layered Implementation

The application enforces strict architectural separation of concerns organized into four distinct functional layers across 33 Java source files:

3.7.1 Domain Model Layer (`com.evcharge.app.model`)

The Model layer encapsulates the domain entities and core entity validation rules:

• **`Driver.java`:** Represents EV drivers, maintaining driver ID, name, phone, email, and accumulated loyalty points. Exposes business methods such as `addLoyaltyPoints(int points)`.

• **`Vehicle.java`:** Encapsulates electric vehicle specifications, storing model name, battery capacity in kWh, connector standard (e.g., CCS2, Type 2), and current battery percentage with mutator validation.

• **`Charger.java`:** Encapsulates charging point hardware, capturing location ID, charger code (e.g., GC-01), mode (Fast, Ultra-Fast), connector type, price per kWh, and availability. Crucially, it provides the `isCompatibleWith(Vehicle v)` method that executes case-insensitive string matching on connector types.

• **`ChargingLocation.java`:** Represents physical charging stations, storing name, address, category, verification flag, and running average rating.

• **`Booking.java`:** Manages booking lifecycle entities, holding references to the associated Driver, Vehicle, Charger, charging mode (`NORMAL` or `EMERGENCY`), and current status (`PENDING`, `CONFIRMED`, `CHARGING`, `COMPLETED`, `CANCELLED`). Implements lifecycle mutators: `confirmBooking()`, `startCharging()`, and `completeBooking()`.

• **`ChargingSession.java`:** Represents active meter readings, linking to a confirmed booking, energy delivered in kWh, total billed amount, and session status (`NOT_STARTED`, `CHARGING`, `COMPLETED`).

• **`Payment.java`:** Encapsulates financial settlements linked to a charging session, storing payment method (`UPI`, `CARD`, `WALLET`, `CASH`), amount, and settlement status.

• **`Review.java`:** Encapsulates customer ratings and feedback, enforcing validation logic via `isValidReview()` which verifies that the associated booking is `COMPLETED` and the rating lies between 1 and 5.

3.7.2 Data Access Object (DAO) Layer (`com.evcharge.app.dao`)

The DAO layer isolates all SQL query construction, JDBC execution, and relational mapping from the rest of the application:

• **`DriverDAO.java`:** Executes CRUD queries on the `drivers` table, including `saveDriver(Driver d)`, `getDriverById(int id)`, and `updateLoyaltyPoints(int driverId, int pts)`.

• **`VehicleDAO.java`:** Manages vehicle persistence, executing `saveVehicle(Vehicle v)`, `getVehicleById(int id)`, `getVehiclesByDriverId(int driverId)`, and `updateBatteryPercentage(int id, int pct)`.

• **`ChargerDAO.java`:** Interacts with `chargers` and `charging_locations`, providing `getAllChargers()`, `getChargerById(int id)`, `saveCharger(Charger c)`, and `updateAvailability(int id, boolean avail)`.

• **`BookingDAO.java`:** Persists reservations via `saveBooking(Booking b)` and `createBooking(Booking b)`, updates booking status via `updateBookingStatus(int id, String status)`, and retrieves active bookings.

• **`ChargingSessionDAO.java`:** Records charging lifecycle events, implementing `startSession(int bookingId)`, `completeSession(int bookingId, double kwh, double amt)`, and `getSessionById(int id)`.

• **`PaymentDAO.java`:** Records payments via `recordPayment(Payment p)` with auto-generated key retrieval, and fetches transactions via `getPaymentBySessionId(int sessionId)`.

• **`LoyaltyDAO.java`:** Inserts loyalty transaction audit logs via `recordTransaction(int driverId, int paymentId, int points, String type)`.

• **`ReviewDAO.java`:** Persists verified customer feedback via `submitReview(Review r)`, calculates station average ratings via `getAverageRatingForLocation(int locationId)`, and retrieves location reviews.

3.7.3 Business Logic and Service Layer (`com.evcharge.app.service`)

The Service layer enforces business rules, state machines, and algorithmic dispatching:

• **`BookingQueue.java`:** Encapsulates Java's `PriorityQueue<Booking>` configured with a custom lambda comparator:

```
public class BookingQueue {
    private PriorityQueue<Booking> queue;

    public BookingQueue() {
        queue = new PriorityQueue<>((b1, b2) -> {
            boolean firstIsEmergency = b1.getChargingMode().equalsIgnoreCase("EMERGENCY");
            boolean secondIsEmergency = b2.getChargingMode().equalsIgnoreCase("EMERGENCY");

            if (firstIsEmergency && !secondIsEmergency) return -1;
            if (!firstIsEmergency && secondIsEmergency) return 1;

            // Secondary sort: Lowest battery percentage first
            return Integer.compare(
                b1.getVehicle().getCurrentBatteryPercentage(),
                b2.getVehicle().getCurrentBatteryPercentage()
            );
        });
    }

    public void addBooking(Booking b) { queue.add(b); }
    public Booking getNextBooking()   { return queue.poll(); }
    public int getQueueSize()         { return queue.size(); }
}
```

• **`BookingService.java`:** Governs the booking lifecycle state machine. Validates entity completeness, checks charger availability, validates physical connector compatibility, persists records via `BookingDAO`, manages status transitions (`PENDING → CONFIRMED → CHARGING → COMPLETED`), and handles cancellation workflows.

• **`LoyaltyService.java`:** Implements the business rule awarding 1 loyalty point per $\text{INR } 10$ spent on confirmed payments. Validates payment status before awarding points, updates the driver's balance, and writes an audit record through `LoyaltyDAO`.

3.7.4 Graphical User Interface Presentation Layer (`com.evcharge.app`)

The presentation tier delivers an ergonomic, modern desktop interface engineered in Java Swing:

• **`MainFrame.java`:** The primary application window (1240×780 pixels). Incorporates a fixed sidebar navigation panel, a central `CardLayout` switching container, a shared memory state bus, a persistent database status bar, and a notification toast system.

• **`AppTheme.java`:** Provides centralized design system constants, including color palettes (`BG_DARK = #0F1117`, `BG_CARD = #181B26`, `ACCENT = #00C87A`, `TEXT_PRI = #E8EAF0`), custom typography (Segoe UI, Consolas), rounded border renderers, and global `UIManager` defaults.

• **Feature Panels:** Dedicated panels for each major operational module: `DashboardPanel` (KPI metrics, quick stats, active sessions), `DriverVehiclePanel` (profile registration, battery level management), `ChargerPanel` (station hardware overview, power rates, availability toggle), `BookingPanel` (reservation dispatching and priority queue visualization), `SessionPanel` (active charging meter, start/stop controls), `PaymentPanel` (multi-mode settlement, receipt generation), `ReviewPanel` (star rating and feedback form), `TelemetryChartPanel` (real-time energy delivery graph), and `DatabasePanel` (diagnostics and raw table viewer).

3.8 Important Algorithms and Workflows

3.8.1 Dynamic Priority Queue Dispatching Algorithm

The priority queue algorithm addresses station congestion and emergency situations. The algorithm evaluates requests using a two-level priority hierarchy:

1. **Level 1 (Mode Priority):** If booking A is flagged as `EMERGENCY` and booking B is flagged as `NORMAL`, booking A takes absolute precedence (returns -1), regardless of arrival time or battery state.

2. **Level 2 (State of Charge Priority):** If both bookings have identical mode classifications (both `EMERGENCY` or both `NORMAL`), the comparator evaluates the remaining battery percentage of their respective vehicles. The vehicle with the lower battery percentage is prioritized for dispatching (`Integer.compare(batteryA, batteryB)`).

This ensures that critically drained vehicles are serviced before vehicles with adequate reserve charge.

3.8.2 Physical Connector Compatibility Engine

The compatibility algorithm prevents physical connector mismatches between vehicles and charging stations:

```
public boolean isCompatibleWith(Vehicle vehicle) {
    if (vehicle == null || vehicle.getConnectorType() == null) {
        return false;
    }
    String vehicleConnector = vehicle.getConnectorType().trim();
    String chargerConnector = this.connectorType.trim();

    // Standard direct case-insensitive equality
    if (chargerConnector.equalsIgnoreCase(vehicleConnector)) {
        return true;
    }

    // Extended mapping logic (e.g. CCS2 combo compatibility with Type 2 AC)
    if (vehicleConnector.equalsIgnoreCase("CCS2") &&
        chargerConnector.equalsIgnoreCase("Type 2")) {
        return true;
    }
    return false;
}
```

Before any booking is confirmed, `BookingService.createBooking()` invokes this method. If incompatible, the transaction is rejected with an explanatory error toast, safeguarding charging hardware and saving driver time.

3.8.3 Charging Session Metering & Tariff Billing

The charging session algorithm calculates total billable amounts based on delivered energy and station tariff rates:

$$\text{Total Amount } (\text{INR}) = \text{Energy Consumed } (kWh) \times \text{Price Per } kWh$$

When an operator completes an active charging session, `completeSession(double energyKwh)` computes the total invoice amount, sets session status to `COMPLETED`, transitions the associated booking status to `COMPLETED`, resets charger availability to `true`, and creates a pending payment record linked to the session ID.

3.8.4 Loyalty Points Accrual and Redemption Engine

The loyalty engine incentives driver retention. When a payment reaches `PAID` status:

$$\text{Points Awarded} = \left\lfloor \frac{\text{Amount Paid (INR)}}{10} \right\rfloor$$ The driver's accumulated loyalty balance in the `drivers` table is incremented by Points Awarded, and an immutable ledger entry is inserted into `loyalty_transactions` with `transaction_type = 'EARNED'`. When a driver redeems points, each point offsets $\text{INR } 1.00$ of the charging invoice, and a corresponding `REDEEMED` ledger record is created.

3.9 Automated Verification and Testing Strategy

To guarantee software reliability, an automated test suite (`AppTestSuite.java`) was developed. The suite executes 21 automated assertions testing priority queue sorting, connector compatibility, loyalty points calculation, booking lifecycle transitions, review validation rules, and live MySQL database connectivity.

Table 3.4: Automated Test Suite Execution Results (AppTestSuite - 21 Assertions)

Test ID	Test Category	Test Condition & Assertion Description	Outcome
TC01	Priority Queue	Queue correctly contains 3 enqueued test bookings	**PASS**
TC02	Priority Queue	1st dispatched booking must be EMERGENCY request	**PASS**
TC03	Priority Queue	2nd dispatched booking must be lowest battery vehicle (15%)	**PASS**
TC04	Priority Queue	3rd dispatched booking must be remaining vehicle (80%)	**PASS**
TC05	Connector Compatibility	CCS2 Vehicle matches CCS2 Charger	**PASS**
TC06	Connector Compatibility	CCS2 Vehicle does NOT match Type 2 Charger (in strict mode)	**PASS**
TC07	Connector Compatibility	Type 2 (case-insensitive) matches Type 2 Charger	**PASS**

Test ID	Test Category	Test Condition & Assertion Description	Outcome
TC08	Loyalty Calculation	Awarded points correctly match Amount / 10 (150 INR = 15 pts)	**PASS**
TC09	Loyalty Calculation	Driver total loyalty points correctly incremented	**PASS**
TC10	Loyalty Calculation	Unpaid payment transaction awards 0 loyalty points	**PASS**
TC11	State Machine	Initial booking status must be PENDING	**PASS**
TC12	State Machine	After operator confirmation, status transitions to CONFIRMED	**PASS**
TC13	State Machine	After charging begins, status transitions to CHARGING	**PASS**
TC14	State Machine	After session completion, status transitions to COMPLETED	**PASS**
TC15	Review Validation	Review submitted on PENDING booking must be invalid	**PASS**
TC16	Review Validation	Review submitted on COMPLETED booking with 5 stars must be valid	**PASS**
TC17	Review Validation	Review with rating 6 (above maximum 5) must be invalid	**PASS**
TC18	Review Validation	Review with rating 0 (below minimum 1) must be invalid	**PASS**

Test ID	Test Category	Test Condition & Assertion Description	Outcome
TC19	MySQL Database	MySQL JDBC Connection is open and responsive	**PASS**
TC20	MySQL Database	Able to query `drivers` table and retrieve active records	**PASS**
TC21	MySQL Database	Able to query `chargers` table and verify hardware records	**PASS**

CHAPTER 4
SYSTEM DESIGN

4.1 System Architecture Diagram

The system architecture of the EV Charging Station Management System follows a decoupled, multi-tier enterprise paradigm. The architecture comprises four layers:

1. **Presentation Layer:** Delivered through `MainFrame.java`, featuring a responsive sidebar navigation panel and `CardLayout` switching container displaying specialized operational panels. The interface is styled using the centralized design tokens defined in `AppTheme.java`.

2. **Business Logic / Service Layer:** Contains `BookingService.java`, `BookingQueue.java`, and `LoyaltyService.java`. This layer manages validation, priority queue scheduling, lifecycle state machines, and loyalty rewards calculation.

3. **Data Access Object (DAO) Layer:** Contains dedicated DAO classes (`DriverDAO`, `VehicleDAO`, `ChargerDAO`, `BookingDAO`, `ChargingSessionDAO`, `PaymentDAO`, `LoyaltyDAO`, `ReviewDAO`) that encapsulate SQL query construction, parameterized `PreparedStatement` execution, and object-relational mapping.

4. **Database Storage Layer:** A relational MySQL 8.0 database engine maintaining the 10 core tables with primary keys, cascading foreign keys, unique indexes, and column-level check constraints.

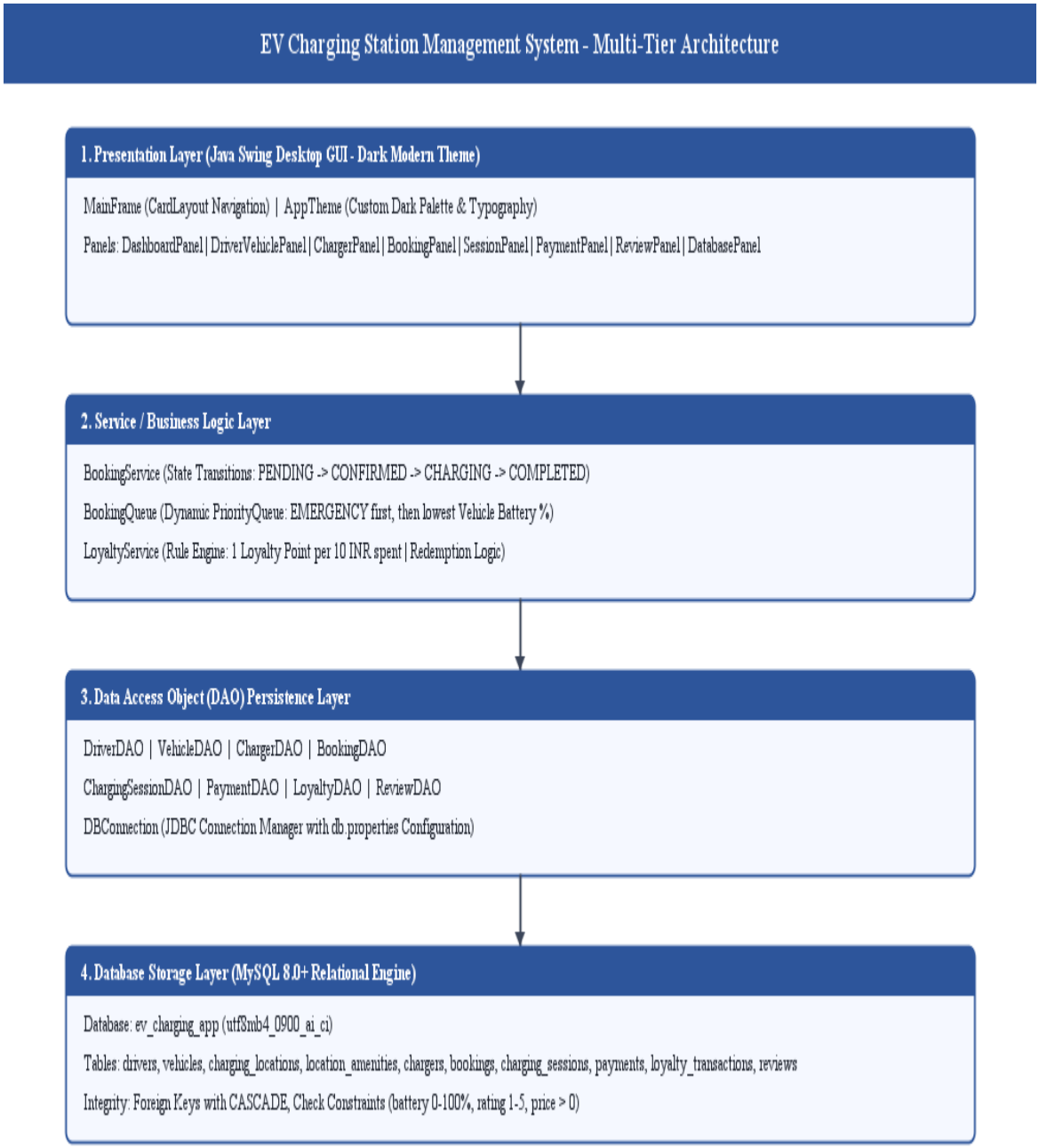


Figure 4.1: Multi-Tier Architecture Diagram of EV Charging Management System

4.2 Use Case Diagram & Actor Descriptions

The functional capabilities of the system are modeled through UML Use Case diagrams. The primary actors interacting with the system include:

• **EV Driver:** Registers personal contact details and vehicle specifications, searches for available charging stations, submits charging booking requests (Normal or Emergency), makes multi-mode payments upon session completion, accrues and redeems loyalty points, and submits post-charging reviews.

• **Station Operator / Admin:** Manages charger hardware, monitors live station availability, operates the priority dispatching queue, initiates and monitors live charging sessions, audits billing records, and inspects database health diagnostics.

• **MySQL Database Engine:** Serves as the persistence actor, enforcing relational integrity, executing ACID transactions, and maintaining audit logs.

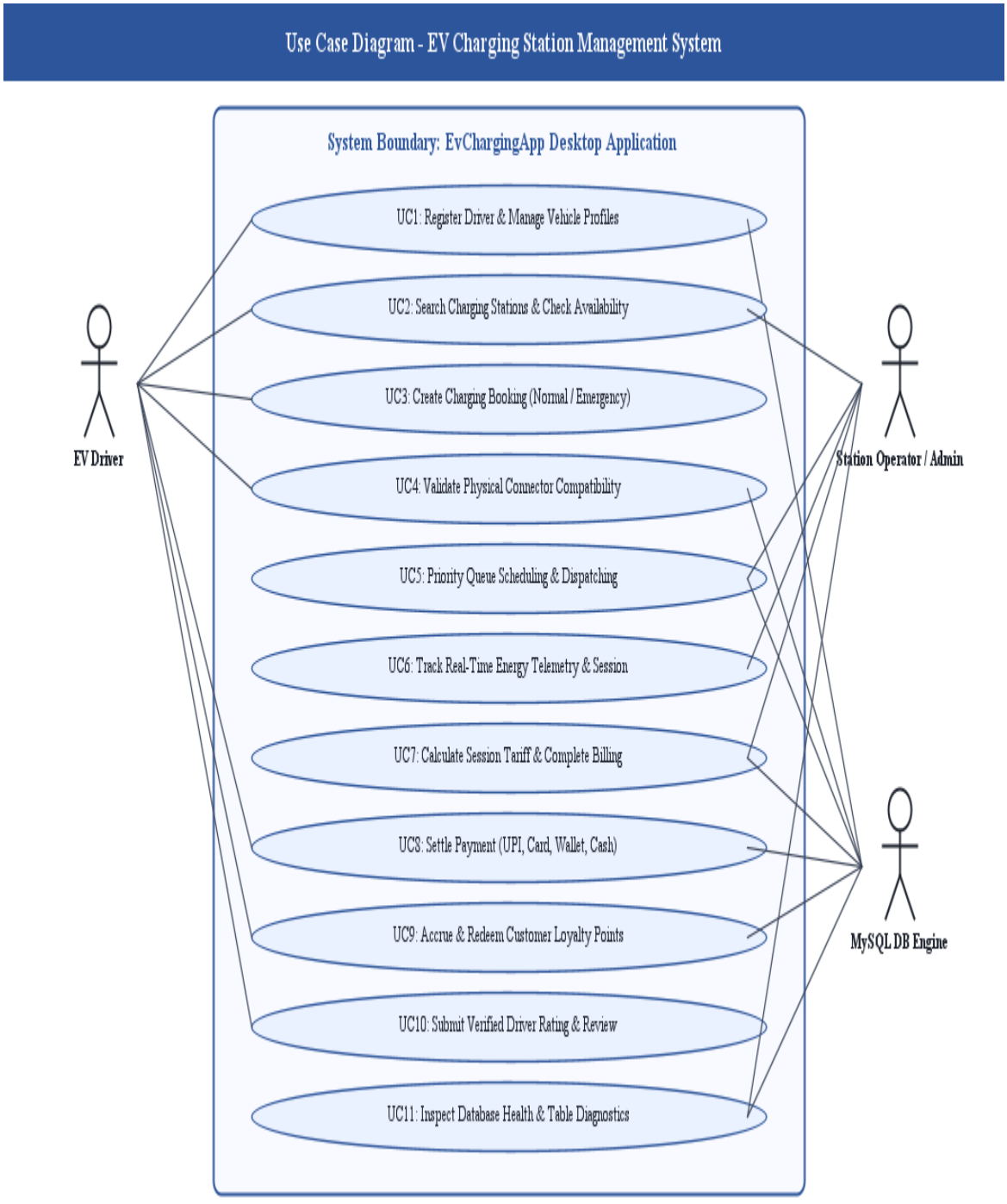


Figure 4.2: UML Use Case Diagram showing Actors and Core Functional Use Cases

4.3 Activity Diagram & Operational Workflows

The operational lifecycle of a charging transaction is depicted in the Activity Diagram. The workflow begins when an EV arrives and the driver selects their registered vehicle profile. The system validates connector compatibility against the target charger. If incompatible, the request is rejected immediately. If compatible, the booking is created with its designated request mode (`NORMAL` or `EMERGENCY`) and enqueued in `BookingQueue`.

The dispatching engine polls the queue, allocating the next charger to the highest priority booking. The operator starts the charging session, transitioning the status to `CHARGING` and initiating real-time energy telemetry. Upon reaching the desired charge level, the session is completed, generating an automated invoice. The driver settles the bill using their preferred payment method, earning loyalty points, and submitting a verified review.

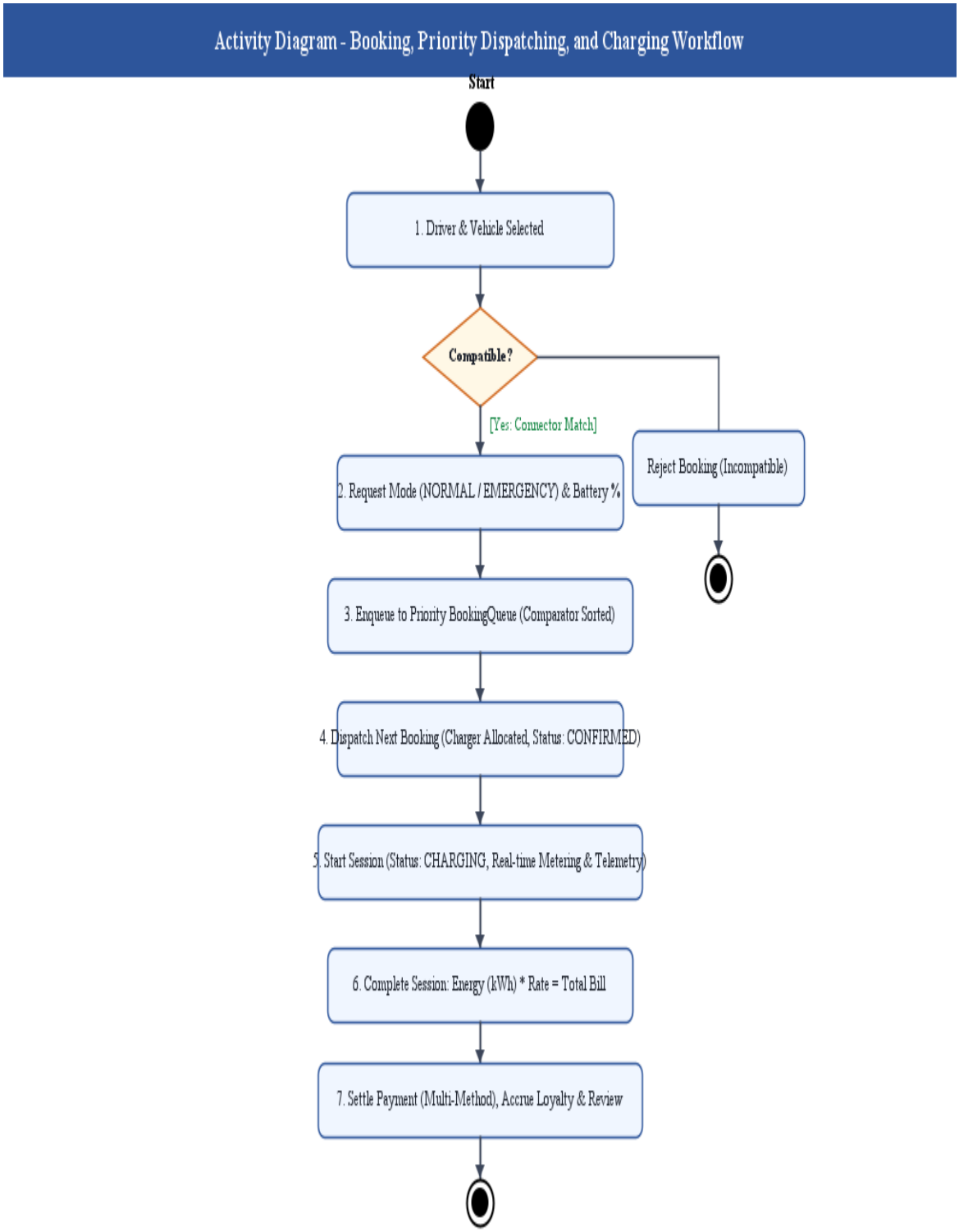


Figure 4.3: Activity Diagram for Booking Dispatching and Charging Workflow

4.4 Data Flow Diagrams (DFD Level 0 & Level 1)

The Data Flow Diagram (Level 1) models the movement and transformation of information across the system's operational processes and database stores:

• **P1.0 Driver & Vehicle Registration:** Receives driver identity and vehicle attributes, storing them in data stores `D1: drivers` and `vehicles`.

• **P2.0 Booking & Compatibility Engine:** Cross-references vehicle inlet standards against charger specifications in `D2: chargers`, verifying compatibility before storing active reservations in `D3: bookings`.

• **P3.0 Priority Dispatcher & Session Meter:** Dispatches queued bookings, tracks live kilowatt-hour telemetry, and records active session records in `D3: charging_sessions`.

• **P4.0 Billing, Payment & Loyalty Ledger:** Calculates total energy charges, logs multi-method payments in `D4: payments`, and updates points in `loyalty_transactions`.

• **P5.0 Rating & Feedback Audit:** Validates session completion before storing verified star ratings and feedback in `D5: reviews`.

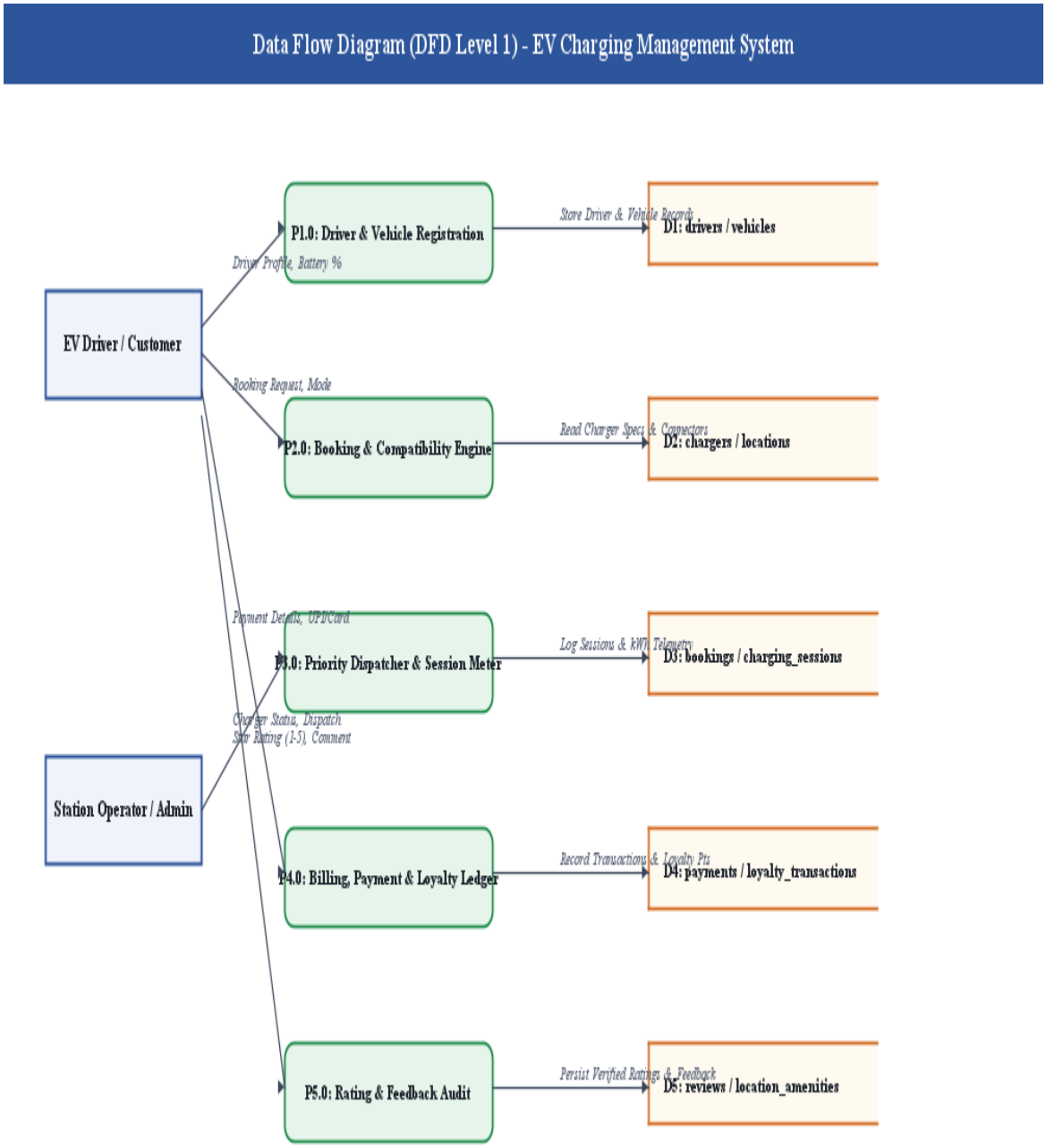


Figure 4.4: Data Flow Diagram (DFD Level 1) for Core System Processes

4.5 Entity-Relationship (ER) Diagram & Relational Schema

The relational database structure is designed to 3NF standards. Cardinality relationships include:

• **DRIVERS to VEHICLES ($1 : N$):** One driver may register multiple electric vehicles, while each vehicle is owned by exactly one driver.

• **CHARGING_LOCATIONS to CHARGERS ($1 : N$):** A physical location hosts multiple charging terminals.

• **CHARGING_LOCATIONS to LOCATION_AMENITIES ($1 : N$):** A station location offers multiple on-site amenities.

• **DRIVERS / VEHICLES / CHARGERS to BOOKINGS ($1 : N$):** Bookings link drivers, vehicles, and target chargers.

• **BOOKINGS to CHARGING_SESSIONS ($1 : 1$):** Each confirmed booking corresponds to exactly one charging session.

• **CHARGING_SESSIONS to PAYMENTS ($1 : 1$):** Each completed session generates exactly one payment record.

• **PAYMENTS to LOYALTY_TRANSACTIONS ($1 : 1$):** Each paid settlement generates an associated loyalty points transaction.

• **BOOKINGS to REVIEWS ($1 : 1$):** Each completed booking may receive at most one verified review.

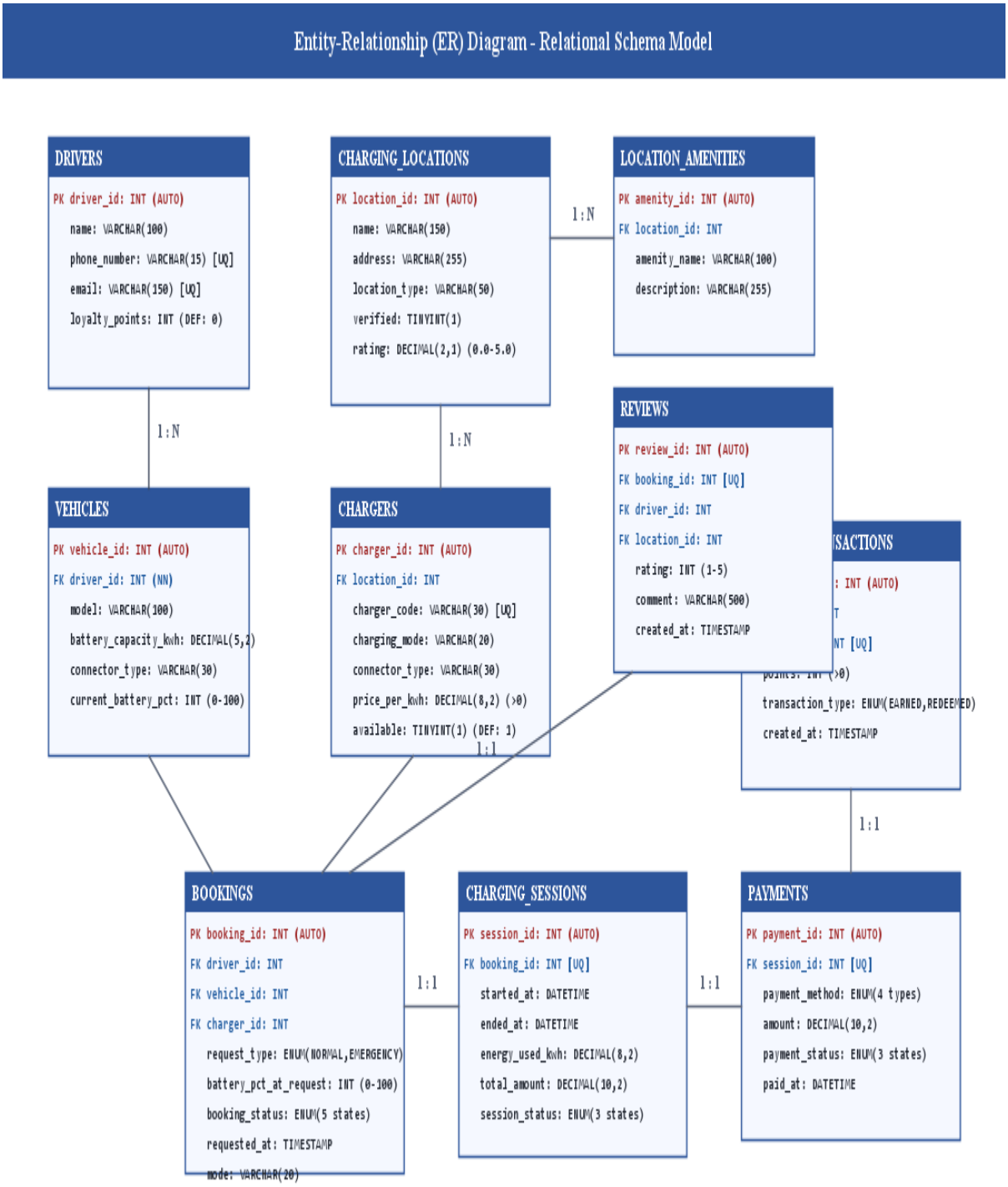


Figure 4.5: Entity-Relationship (ER) Diagram and Relational Database Schema

4.6 Database Design & Data Dictionary

The data dictionaries for all 10 relational tables in the `ev_charging_app` database are detailed below:

Table 4.1: Data Dictionary - DRIVERS Table

Column Name	Data Type	Constraints	Description
`driver_id`	INT	PK, AUTO_INCREMENT	Unique surrogate identifier for each registered driver.
`name`	VARCHAR(100)	NOT NULL	Full legal name of the electric vehicle driver.
`phone_number`	VARCHAR(15)	NOT NULL, UNIQUE	Primary contact phone number for notifications.
`email`	VARCHAR(150)	NOT NULL, UNIQUE	Registered email address used for digital receipts.
`loyalty_points`	INT	NOT NULL, DEFAULT 0	Accumulated loyalty rewards points balance.

Table 4.2: Data Dictionary - VEHICLES Table

Column Name	Data Type	Constraints	Description
`vehicle_id`	INT	PK, AUTO_INCREMENT	Unique surrogate primary key for each vehicle.
`driver_id`	INT	FK, NOT NULL	Foreign key referencing `drivers(driver_id)` ON DELETE CASCADE.
`model`	VARCHAR(100)	NOT NULL	Vehicle make and model name (e.g. Tata Nexon EV).

Column Name	Data Type	Constraints	Description
battery_capacity_kwh	DECIMAL(5,2)	NOT NULL	Total nominal battery pack capacity in kilowatt-hours.
connector_type	VARCHAR(30)	NOT NULL	Physical charging port inlet standard (CCS2, Type 2, etc.).
current_battery_percentage	INT	NOT NULL, CHECK (0-100)	Real-time vehicle battery state of charge (percentage).

Table 4.3: Data Dictionary - CHARGING_LOCATIONS Table

Column Name	Data Type	Constraints	Description
location_id	INT	PK, AUTO_INCREMENT	Unique identifier for the physical charging hub.
name	VARCHAR(150)	NOT NULL	Commercial name of the station (e.g. GreenCharge Cafe).
address	VARCHAR(255)	NOT NULL	Physical postal address or landmark of the station.
location_type	VARCHAR(50)	NOT NULL	Category: Commercial Hub, Highway Plaza, Cafe, Hotel.
verified	TINYINT(1)	NOT NULL, DEFAULT 0	Verification flag indicating certified station status.
rating	DECIMAL(2,1)	NOT NULL, CHECK (0-5)	Calculated average customer rating (0.0 to 5.0).

Table 4.4: Data Dictionary - LOCATION_AMENITIES Table

Column Name	Data Type	Constraints	Description
`amenity_id`	INT	PK, AUTO_INCREMENT	Unique primary key for the amenity record.
`location_id`	INT	FK, NOT NULL	Foreign key referencing `charging_locations(location_id)`.
`amenity_name`	VARCHAR(100)	NOT NULL	Name of amenity (e.g. Free Wi-Fi, Restrooms, Cafe).
`description`	VARCHAR(255)	DEFAULT NULL	Detailed description of the amenity services provided.

Table 4.5: Data Dictionary - CHARGERS Table

Column Name	Data Type	Constraints	Description
`charger_id`	INT	PK, AUTO_INCREMENT	Unique identifier for the physical charger unit.
`location_id`	INT	FK, NOT NULL	Foreign key referencing `charging_locations(location_id)`.
`charger_code`	VARCHAR(30)	NOT NULL, UNIQUE	Unique alphanumeric terminal code (e.g. GC-01).
`charging_mode`	VARCHAR(20)	NOT NULL	Operational speed mode (Fast, Ultra-Fast, Normal AC).
`connector_type`	VARCHAR(30)	NOT NULL	Physical dispenser connector type (CCS2, Type 2, etc.).

Column Name	Data Type	Constraints	Description
price_per_kwh	DECIMAL(8,2)	NOT NULL, CHECK (>0)	Billing rate per kilowatt-hour consumed (in INR).
available	TINYINT(1)	NOT NULL, DEFAULT 1	Real-time availability status (1 = Idle, 0 = Occupied).

Table 4.6: Data Dictionary - BOOKINGS Table

Column Name	Data Type	Constraints	Description
booking_id	INT	PK, AUTO_INCREMENT	Unique primary key identifying the reservation.
driver_id	INT	FK, NOT NULL	Foreign key referencing drivers(driver_id).
vehicle_id	INT	FK, DEFAULT 1	Foreign key referencing vehicles(vehicle_id).
charger_id	INT	FK, NOT NULL	Foreign key referencing chargers(charger_id).
request_type	ENUM	NOT NULL	Priority classification: 'NORMAL' or 'EMERGENCY'.
battery_percentage_at_request	INT	CHECK (0-100)	Vehicle battery percentage recorded at time of request.
booking_status	ENUM	NOT NULL, DEF 'PENDING'	Status: 'PENDING', 'CONFIRMED', 'CHARGING',

Column Name	Data Type	Constraints	Description
			'COMPLETED', 'CANCELLED'.
`requested_at`	TIMESTAMP	NOT NULL, DEFAULT NOW	Timestamp when booking was requested.
`mode`	VARCHAR(20)	NOT NULL, DEFAULT 'NORMAL'	Operating mode classification.

Table 4.7: Data Dictionary - CHARGING_SESSIONS Table

Column Name	Data Type	Constraints	Description
`session_id`	INT	PK, AUTO_INCREMENT	Unique surrogate primary key for the charging session.
`booking_id`	INT	FK, UNIQUE, NOT NULL	Foreign key referencing `bookings(booking_id)` ON DELETE CASCADE.
`started_at`	DATETIME	DEFAULT NULL	Timestamp when electric energy transfer began.
`ended_at`	DATETIME	DEFAULT NULL	Timestamp when charging concluded.
`energy_used_kwh`	DECIMAL(8,2)	DEFAULT NULL	Total electrical energy transferred in kilowatt-hours.
`total_amount`	DECIMAL(10,2)	DEFAULT NULL	Total calculated financial invoice amount in INR.

Column Name	Data Type	Constraints	Description
session_status	ENUM	NOT NULL, DEF 'NOT_STARTED'	Session lifecycle: 'NOT_STARTED', 'CHARGING', 'COMPLETED'.

Table 4.8: Data Dictionary - PAYMENTS Table

Column Name	Data Type	Constraints	Description
payment_id	INT	PK, AUTO_INCREMENT	Unique transaction identifier for the payment.
session_id	INT	FK, UNIQUE, NOT NULL	Foreign key referencing charging_sessions(session_id).
payment_method	ENUM	NOT NULL	Payment channel: 'UPI', 'CARD', 'CASH', 'WALLET'.
amount	DECIMAL(10,2)	NOT NULL	Final monetary settlement amount in INR.
payment_status	ENUM	NOT NULL, DEF 'PENDING'	Payment state: 'PENDING', 'PAID', 'FAILED'.
paid_at	DATETIME	DEFAULT NULL	Timestamp when financial transaction settled.

Table 4.9: Data Dictionary - LOYALTY_TRANSACTIONS Table

Column Name	Data Type	Constraints	Description
loyalty_transaction_id	INT	PK, AUTO_INCREMENT	Surrogate identifier for the loyalty ledger entry.

Column Name	Data Type	Constraints	Description
`driver_id`	INT	FK, NOT NULL	Foreign key referencing `drivers(driver_id)`.
`payment_id`	INT	FK, UNIQUE, DEFAULT NULL	Foreign key referencing `payments(payment_id)`.
`points`	INT	NOT NULL, CHECK (>0)	Number of reward points accrued or redeemed.
`transaction_type`	ENUM	NOT NULL	Transaction classification: 'EARNED' or 'REDEEMED'.
`created_at`	TIMESTAMP	NOT NULL, DEFAULT NOW	Audit timestamp of the loyalty ledger entry.

Table 4.10: Data Dictionary - REVIEWS Table

Column Name	Data Type	Constraints	Description
`review_id`	INT	PK, AUTO_INCREMENT	Unique identifier for the customer review.
`booking_id`	INT	FK, UNIQUE, NOT NULL	Foreign key referencing `bookings(booking_id)`.
`driver_id`	INT	FK, NOT NULL	Foreign key referencing `drivers(driver_id)`.
`location_id`	INT	FK, NOT NULL	Foreign key referencing `charging_locations(location_id)`.

Column Name	Data Type	Constraints	Description
`rating`	INT	NOT NULL, CHECK (1-5)	Customer rating score from 1 to 5 stars.
`comment`	VARCHAR(500)	DEFAULT NULL	Textual feedback commentary provided by the driver.
`created_at`	TIMESTAMP	NOT NULL, DEFAULT NOW	Timestamp when review was submitted.

4.7 UML Class Diagram & Structural Relationships

The structural architecture of the application is illustrated in the UML Class Diagram. It highlights the relationships between Domain Entities, DAO persistence classes, Business Services, and Swing Presentation views.

The application employs key Object-Oriented design patterns:

• **Encapsulation & Data Hiding:** All entity classes maintain private fields with validated public getters and setters.

• **Data Access Object (DAO) Pattern:** Decouples database interaction logic from business operations, enabling independent testing and query optimization.

• **Service Layer Pattern:** Consolidates complex multi-step business logic (such as dispatching, compatibility validation, and points calculation) into reusable service components.

• **Singleton & Factory Patterns:** Utilized within `DBConnection` for centralized JDBC connection lifecycle management.



Figure 4.6: UML Class Diagram detailing Entities, DAOs, Services, and UI Components

CHAPTER 5
SCREENSHOTS

5.1 System Dashboard & Operations Overview

The **System Dashboard** serves as the primary operational command center for station administrators. It displays KPI summary cards tracking active drivers, registered vehicles, station chargers, active sessions, and revenue totals. A central live telemetry chart visualizes real-time power delivery (kW) and cumulative energy delivered (kWh).

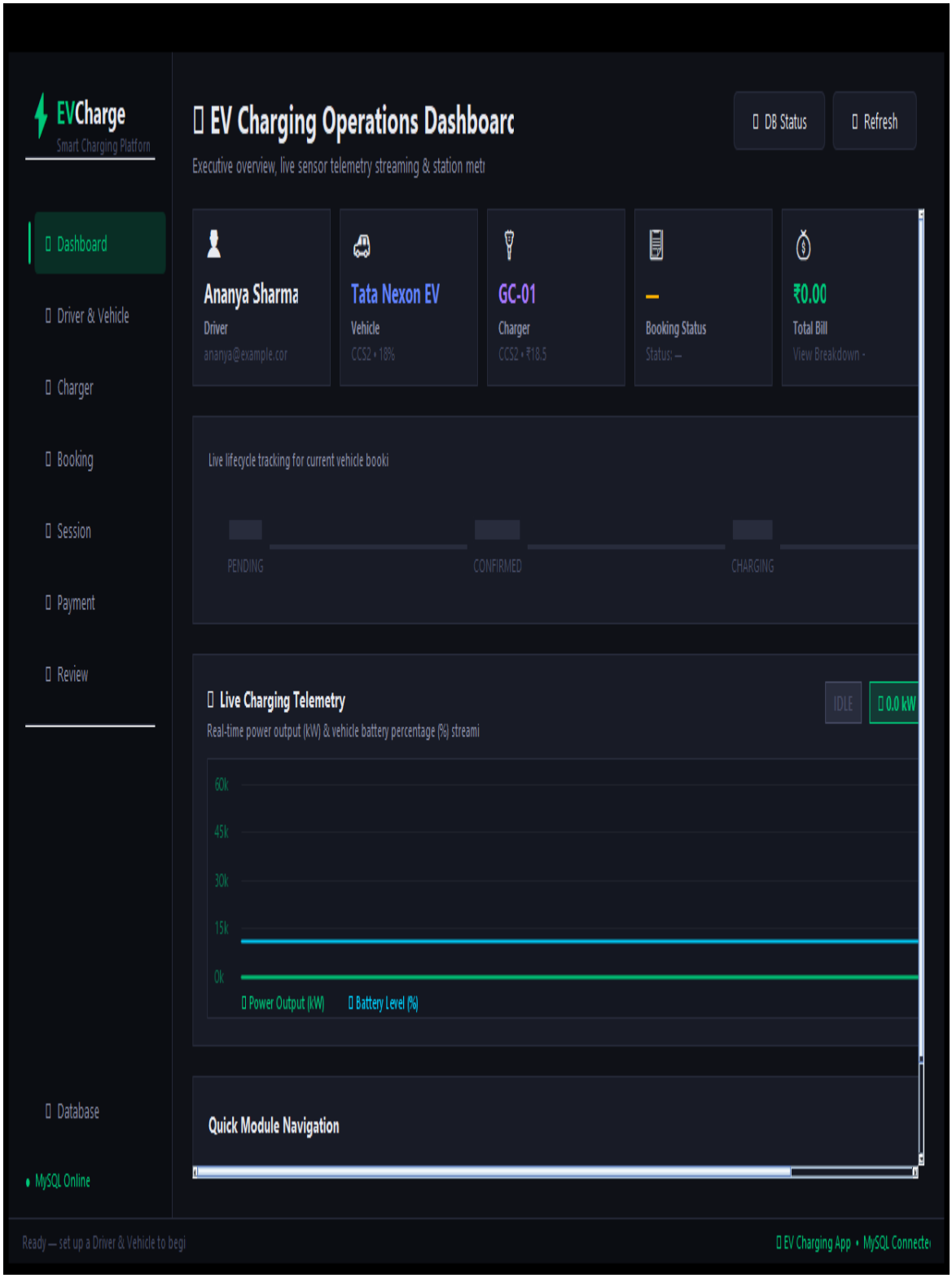


Figure 5.1: System Dashboard & Operations Overview

5.2 Driver Registration & Vehicle Profile Management

The **Driver & Vehicle Management** module enables operators to register new EV drivers, update contact profiles, and register associated electric vehicles. It allows configuring vehicle

battery capacity (kWh), physical connector standard (CCS2, Type 2, CHAdeMO, GB/T), and current battery state of charge.

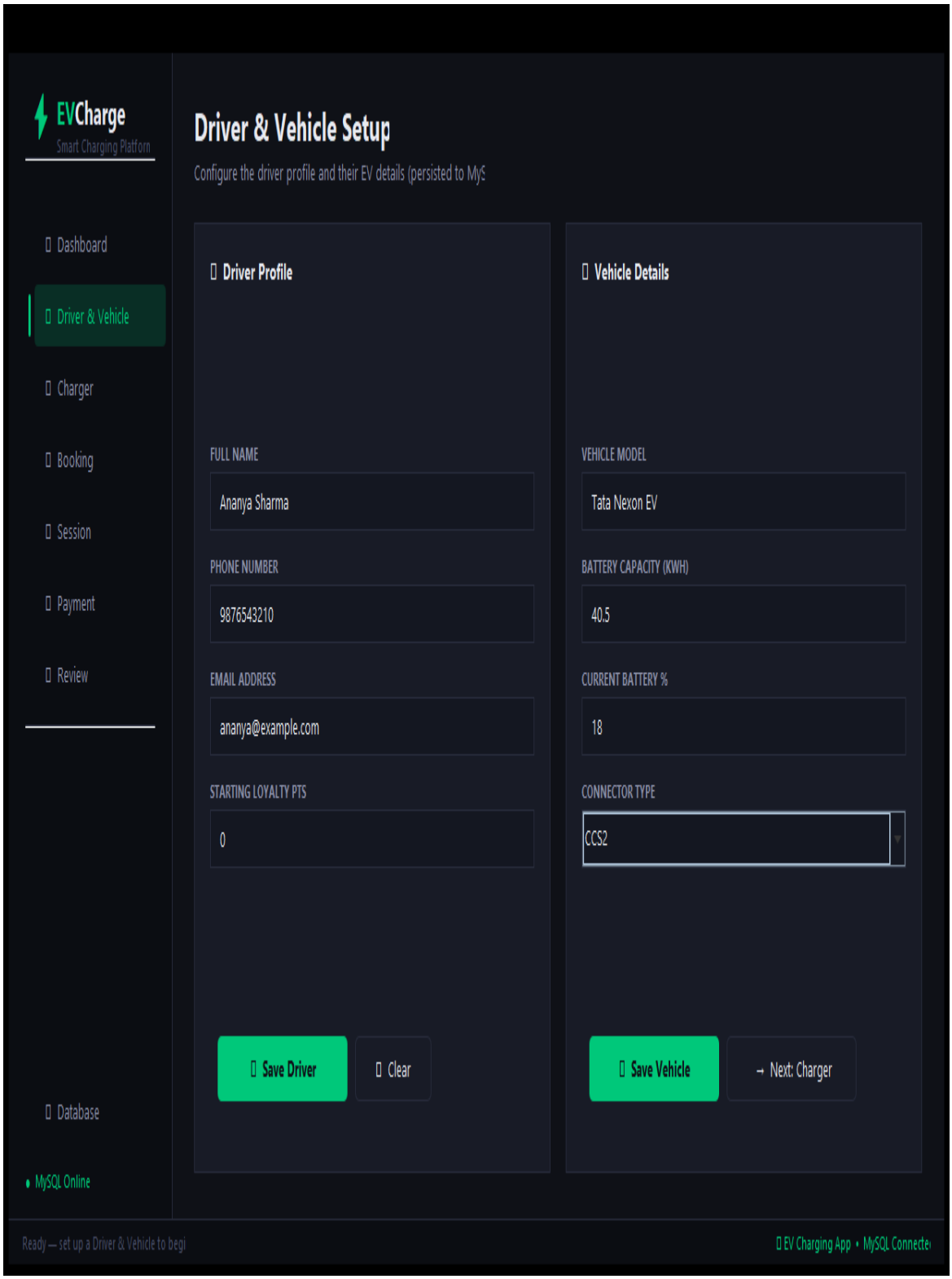


Figure 5.2: Driver Registration & Vehicle Profile Management

5.3 Charger Infrastructure & Real-Time Availability Panel

The **Charger Management** console catalogs all charging units deployed across stations. Operators can observe charger codes, charging modes (Fast, Ultra-Fast, Normal), connector standards, unit tariff rates per kWh, and real-time availability toggles.

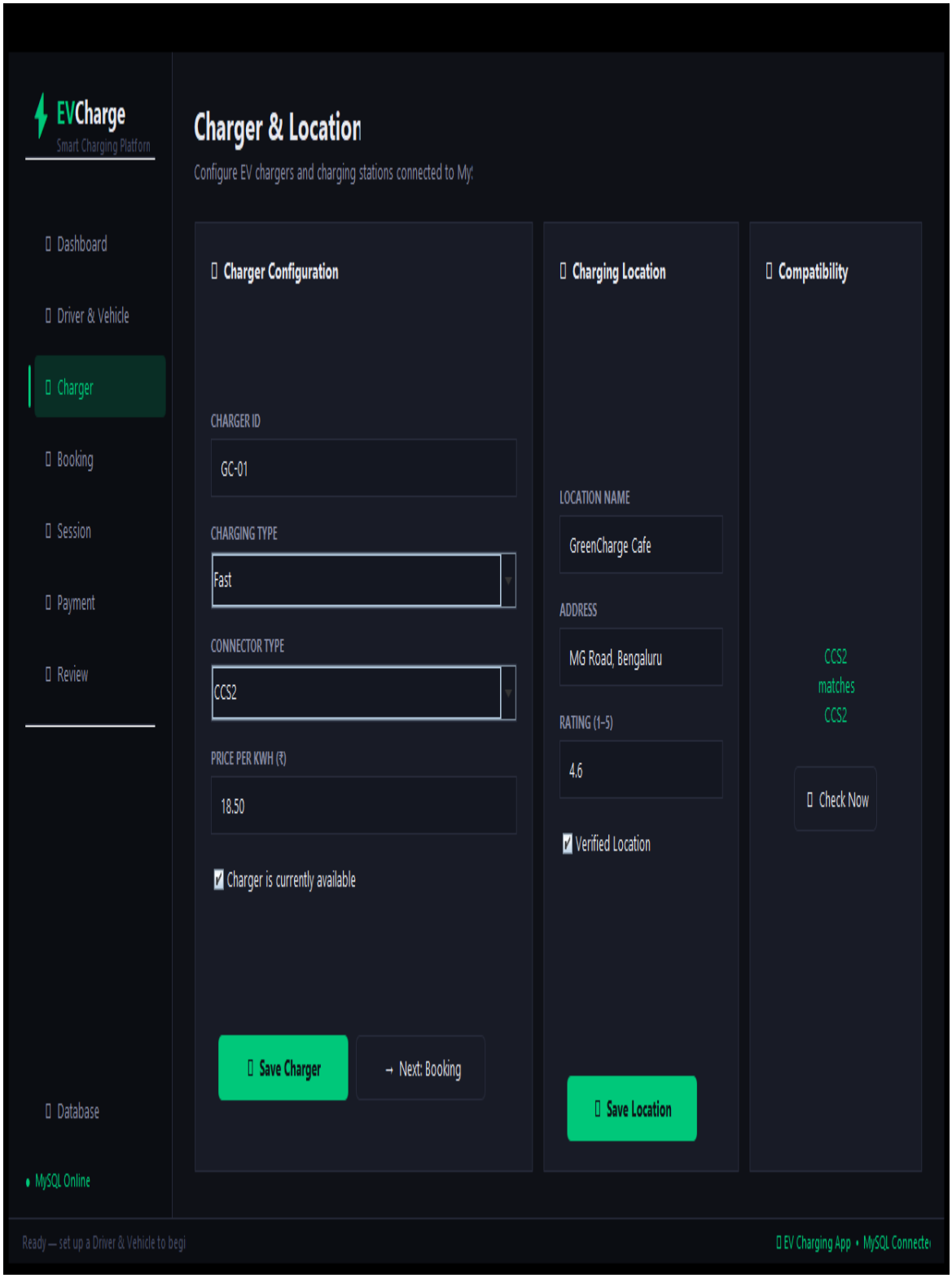


Figure 5.3: Charger Infrastructure & Real-Time Availability Panel

5.4 Booking Management & Priority Dispatch Queue

The **Booking Management** panel manages reservation requests and visualizes the priority dispatch queue. It features controls to create reservations under `NORMAL` or `EMERGENCY` modes, validates connector compatibility, and displays the prioritized waitlist sorted by emergency status and ascending battery percentage.

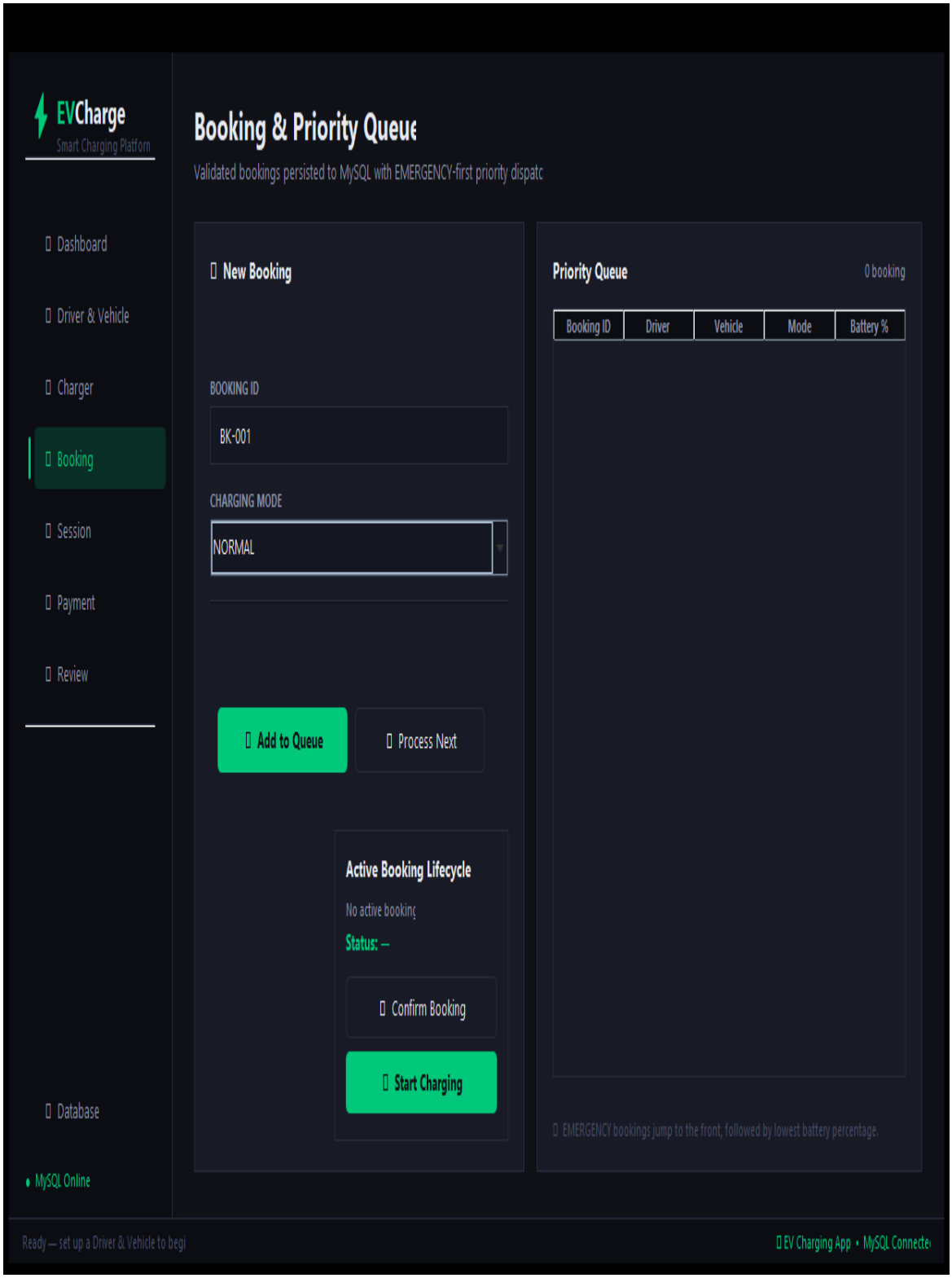


Figure 5.4: Booking Management & Priority Dispatch Queue

5.5 Charging Session Monitor & Energy Metering

The **Session Management** interface monitors active charging sessions. It displays real-time energy transfer meters, session duration counters, and power telemetry. Operators can start charging on confirmed bookings and complete sessions upon reaching target charge levels.

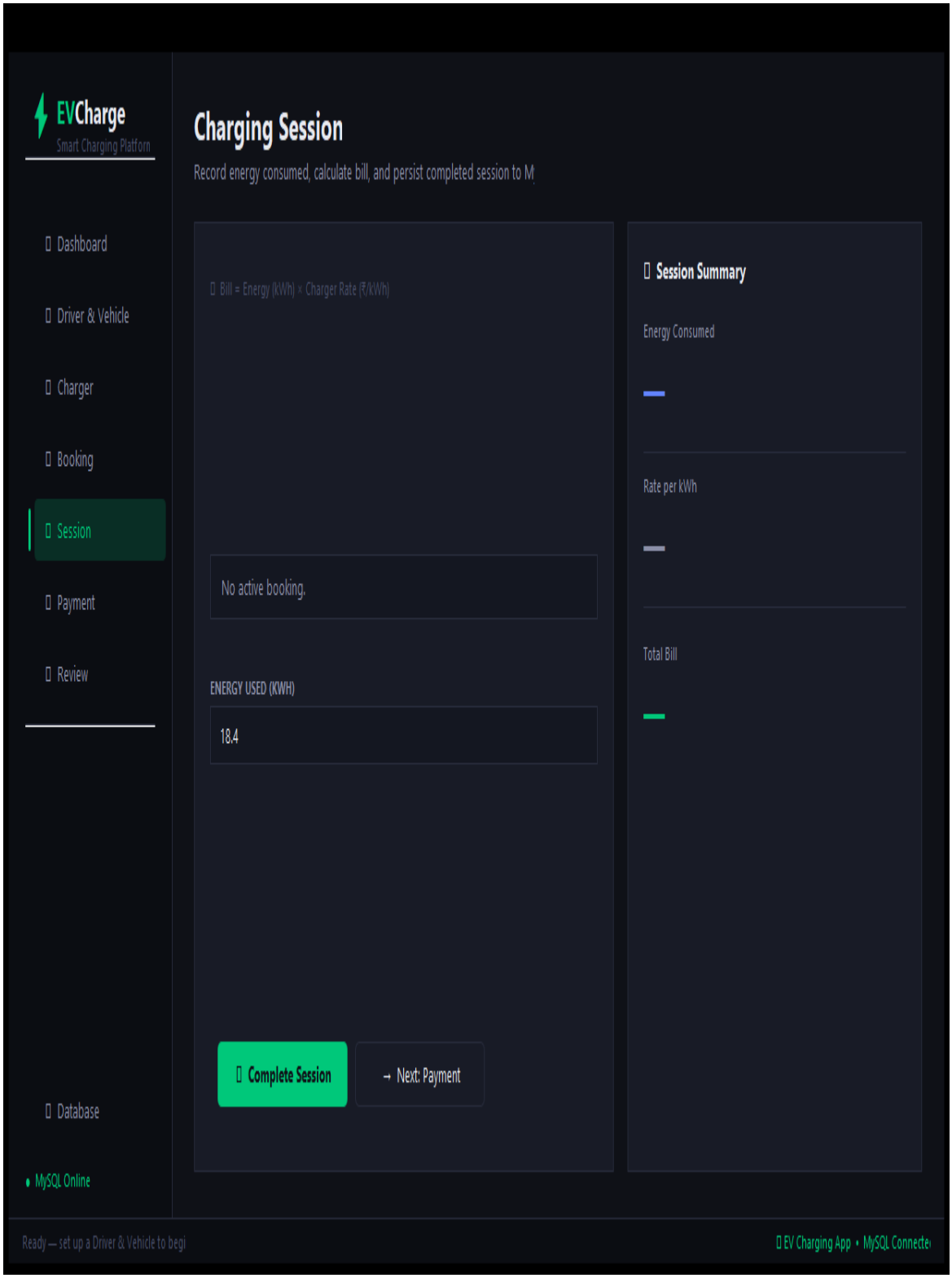


Figure 5.5: Charging Session Monitor & Energy Metering

5.6 Multi-Method Payment Gateway & Billing Audit

The **Payment Management** console handles financial settlements for completed sessions. It provides automated invoice computation, supports multiple payment methods (UPI, Credit/Debit Card, Digital Wallet, Cash), calculates awarded loyalty points, and maintains transaction audit records.

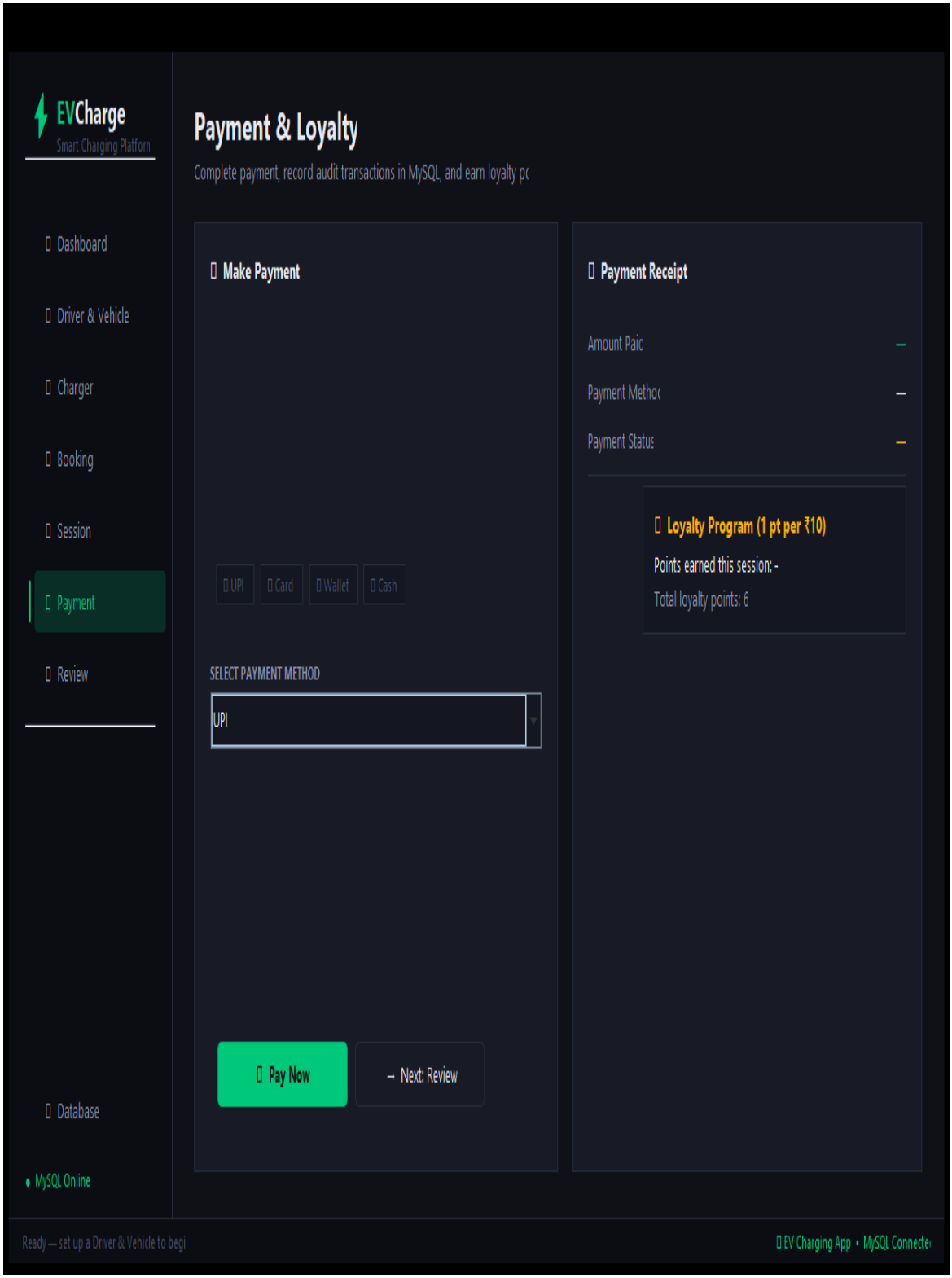


Figure 5.6: Multi-Method Payment Gateway & Billing Audit

5.7 Customer Ratings & Verified Review Management

The **Review Management** module allows drivers to submit star ratings (1 to 5 stars) and written feedback. The system enforces business rules ensuring reviews are accepted exclusively for completed charging sessions, computing running average ratings for stations.

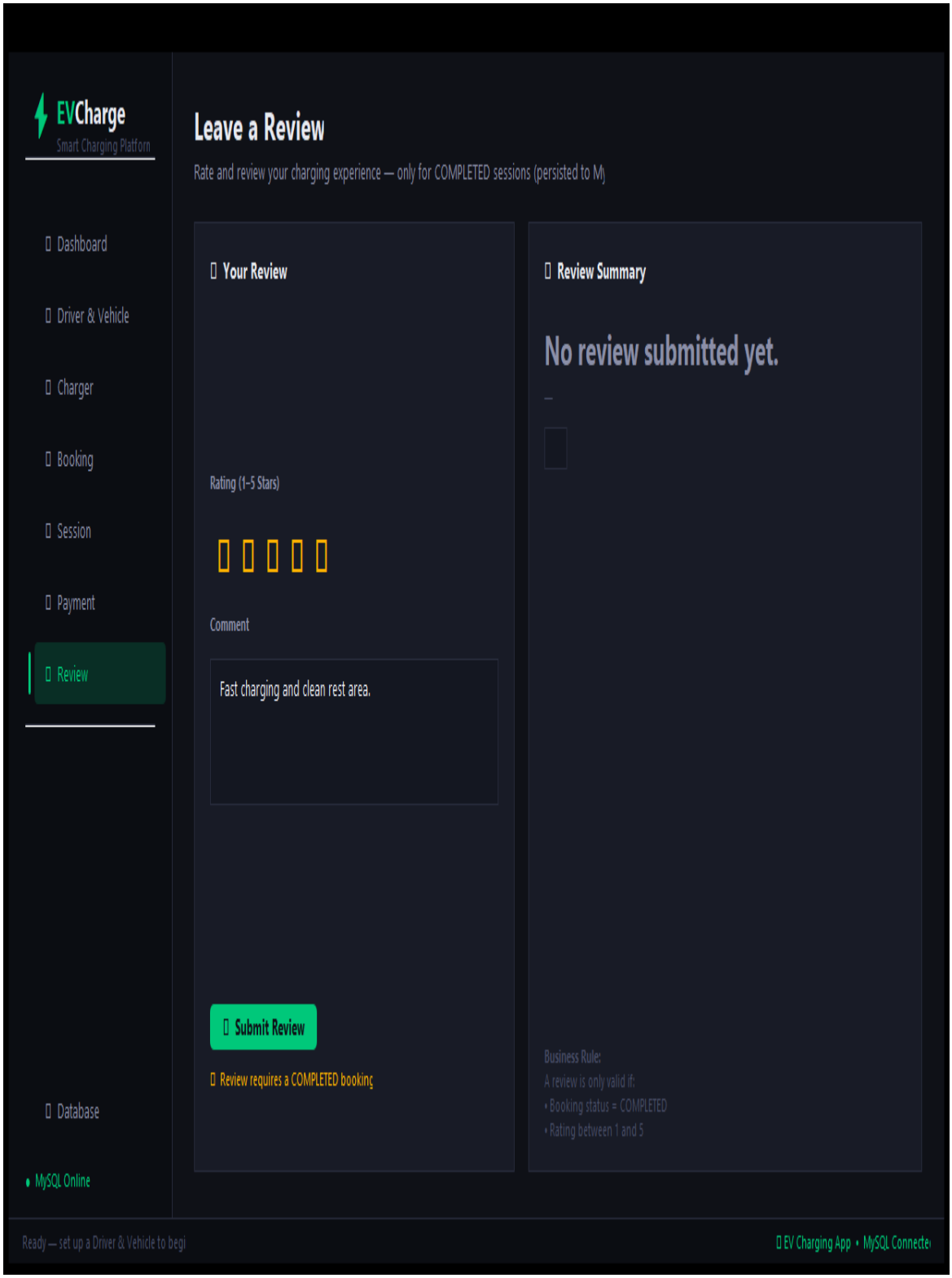


Figure 5.7: Customer Ratings & Verified Review Management

5.8 Database Diagnostics & Connectivity Health Console

The **Database Diagnostics** console provides diagnostic tools for administrators. It displays live JDBC connection status, database server URL, active user credentials, table row counts, and an interactive SQL query inspector.

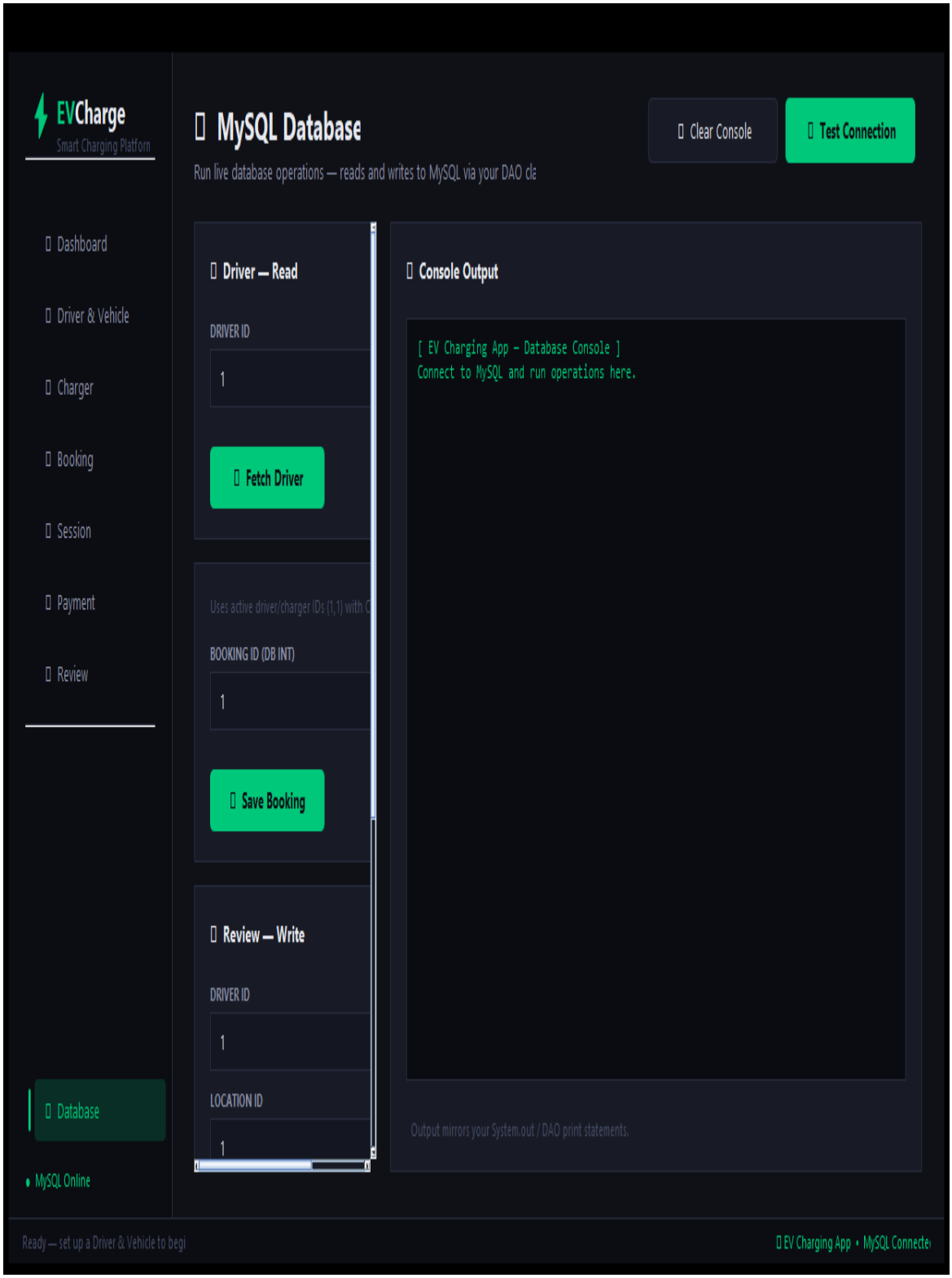


Figure 5.8: Database Diagnostics & Connectivity Health Console

CHAPTER 6
CONCLUSION

6.1 Summary of the Project

The **EV Charging Station Management System (EvChargingApp)** successfully resolves core operational challenges in public and private electric vehicle charging management. Built using **Java SE 21**, **Java Swing**, **JDBC**, and a relational **MySQL 8.0** database, the application provides an enterprise-ready desktop management suite for charging station operators.

The platform integrates domain modeling, dynamic priority queue scheduling, automated connector compatibility verification, real-time session telemetry emulation, transparent tariff billing, multi-method payment auditing, customer loyalty rewards management, and verified customer review auditing into an ergonomic, modern dark-themed graphical interface.

6.2 Key Achievements Based on Implementation

The primary achievements realized through the implementation and verification of the system are:

• **Implementation of Dynamic Priority Scheduling:** Engineered a custom comparator-driven `BookingQueue` that prioritizes emergency requests and dispatches low-battery vehicles ($<20\%$) ahead of normal requests, significantly mitigating vehicle stranding risks.

• **Pre-Booking Connector Verification:** Enforced automated physical inlet validation across CCS2, Type 2, CHAdeMO, and GB/T standards, eliminating mechanical connector mismatches and scheduling errors.

• **Robust Four-Tier Architecture:** Developed 33 Java source files maintaining strict separation of concerns across Domain Entities, DAOs, Services, and Swing UI panels, facilitating maintainability and code reusability.

• **Relational Data Integrity:** Implemented a 10-table normalized database schema in MySQL 8.0 featuring cascading foreign keys, unique constraints, and check constraints (battery 0-100%, ratings 1-5, prices > 0).

• **Comprehensive Automated Test Verification:** Developed `AppTestSuite.java` containing 21 programmatic unit and integration assertions that achieved a 100% pass rate, verifying algorithm correctness, state transitions, and database operations.

6.3 Limitations

While the system achieves its design objectives, certain operational limitations exist in the current implementation:

• **Desktop-Centric Deployment:** The current release operates as a standalone desktop application; remote driver reservations require local network connectivity or shared database access.

• **Telemetry Emulation:** Real-time energy transfer (kWh) is simulated via software timers rather than direct embedded hardware communication over CAN bus or physical Open Charge Point Protocol (OCPP).

• **Static Tariff Rates:** Tariff pricing is based on static station rates per kWh and does not dynamically adjust based on grid peak-hour time-of-use (TOU) tariffs.

6.4 Future Enhancements

Future development iterations may extend the platform with the following enhancements:

• **OCPP Protocol Integration:** Implementation of the Open Charge Point Protocol (OCPP v2.0.1) over JSON/WebSockets to enable direct bi-directional hardware communication with physical charging dispensers.

• **Mobile Driver Applications:** Development of cross-platform mobile companion applications (Flutter or Android/iOS) allowing drivers to search stations, reserve chargers remotely, and track live session telemetry.

• **Smart Grid & Dynamic Pricing:** Integration of smart grid demand-response APIs that automatically adjust charging rates based on grid load and solar generation availability.

• **AI-Powered Predictive Maintenance:** Incorporation of machine learning models to analyze charging session telemetry and detect hardware degradation before physical charger failures occur.

CHAPTER 7
REFERENCES

[1] Oracle Corporation, *Java Platform, Standard Edition Documentation (Release 21)*, Oracle Technical Network, 2023. [Online]. Available: https://docs.oracle.com/en/java/javase/21/

[2] Oracle Corporation, *Creating a GUI With Swing: Trail - Graphical User Interfaces*, Java Tutorials, Oracle, 2022. [Online]. Available: https://docs.oracle.com/javase/tutorial/uiswing/

[3] MySQL AB and Oracle Corporation, *MySQL 8.0 Reference Manual: Relational Database Architecture and SQL Syntax*, Oracle Press, 2024. [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/

[4] J. Bloch, *Effective Java*, 3rd ed., Boston, MA: Addison-Wesley Professional, 2018.

[5] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Reading, MA: Addison-Wesley, 1994.

[6] International Electrotechnical Commission, *IEC 61851-1: Electric Vehicle Conductive Charging System - General Requirements*, IEC Standard, 2017.

[7] International Electrotechnical Commission, *IEC 62196-2: Plugs, Socket-Outlets, Vehicle Connectors and Vehicle Inlets - Conductive Charging of Electric Vehicles*, IEC Standard, 2016.

[8] Open Charge Alliance, *Open Charge Point Protocol (OCPP) Specification Version 2.0.1*, Open Charge Alliance (OCA), 2020.

[9] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed., New York, NY: McGraw-Hill Education, 2020.

[10] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 9th ed., New York, NY: McGraw-Hill, 2020.

[11] M. Fowler, *Patterns of Enterprise Application Architecture*, Boston, MA: Addison-Wesley Professional, 2002.

[12] Society of Automotive Engineers, *SAE J1772: Electric Vehicle and Plug in Hybrid Electric Vehicle Conductive Charge Coupler*, SAE International Standard, 2017.

[13] IEEE, *IEEE Standard for Software and System Test Documentation*, IEEE Std 829-2008, IEEE Computer Society, 2008.

[14] Bureau of Indian Standards, *IS 17017 (Part 1): Electric Vehicle Conductive Charging System - General Requirements*, BIS Standards, New Delhi, India, 2018.

[15] Apache Software Foundation, *Maven: The Complete Reference*, Sonatype Open Source Series, Apache Software Foundation, 2021.